Robot Localization

Javier González Jiménez

Reference Books:

- Probabilistic Robotics. S. Thrun, W. Burgard, D. Fox. MIT Press. 2001
- Simultaneous Localization and Mapping for Mobile Robots: Introduction and Methods. Juan-Antonio Fernández-Madrigal and José Luis Blanco Claraco. IGI-Global. 2013.

Contents

- Introduction
- Least Squares Positioning
- Map registration
  - Landmark points (known correspondence)
  - Scan points: ICP (unknown correspondences)
- Bayes Filtering
  - Grid Filter
  - Kalman Filter
  - Extended Kalman Filter
  - Particle Filter

The robot localization problem

Map of the environment: $\mathcal{M}$

Sensor measurements: $\mathbf{z}_{0:t}$

Robot motions: $\mathbf{u}_{0:t}$

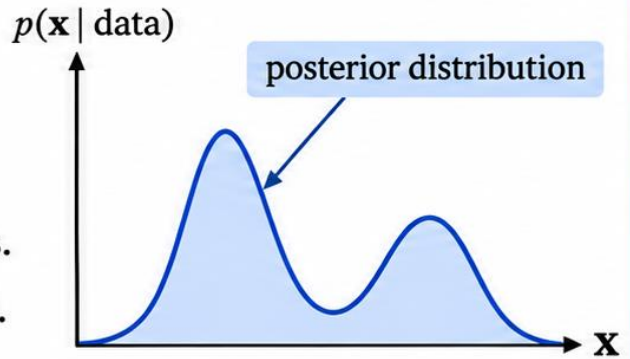
Robot localization

- Point estimation: "best" pose $\hat{\mathbf{x}}$

  OR
- Probabilistic estimation $p(\mathbf{x}_t | \mathbf{z}_{0:t}, \mathbf{u}_{0:t})$

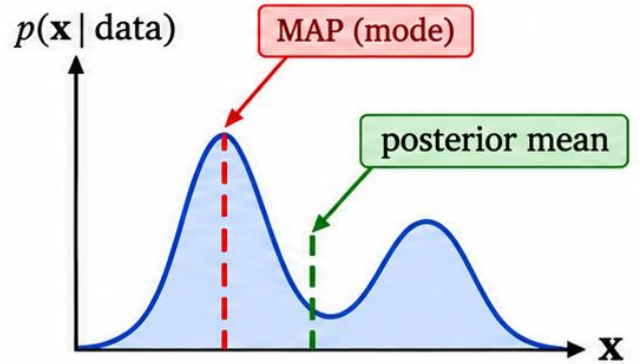
Probabilistic estimation

- Estimates a probability distribution over possible robot poses.
- Represents uncertainty and possibly multiple hypotheses.
- Output: $p(\mathbf{x}_t \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t}, \mathbf{m})$.



Point estimation

- Estimates a single representative robot pose.
- Useful for decision making or control.
- Output: $\hat{\mathbf{x}}_t$.



Typical point estimates from a posterior

$$\hat{\mathbf{x}}_{\text{MAP}} = \arg\max_{\mathbf{x}} p(\mathbf{x} \mid \text{data})$$

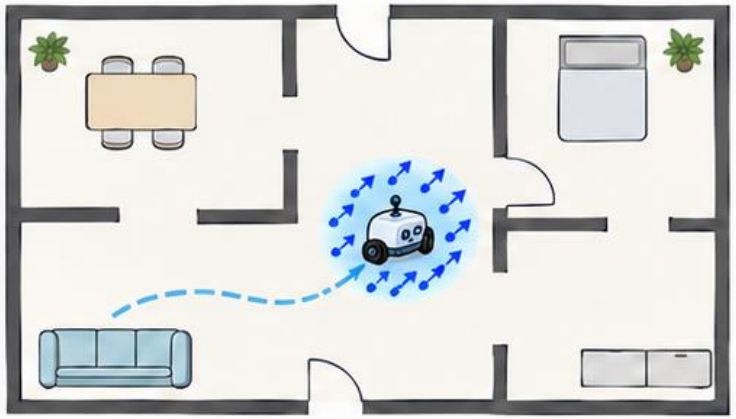
$$\hat{\mathbf{x}}_{\text{mean}} = \mathrm{E}[\mathbf{x} \mid \text{data}]$$

For a Gaussian posterior, mean = mode = median. In general, they may differ.

Three localization problems

What does the robot know about its pose?

1 Position tracking



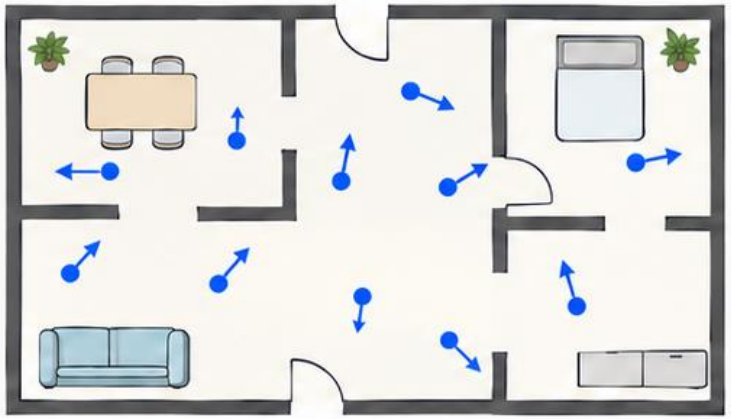
Approximately known pose

Goal: Update pose as the robot moves.

Challenge: Correct drift using sensor data.

Example: Starting from a known docking station.

2 Global localization



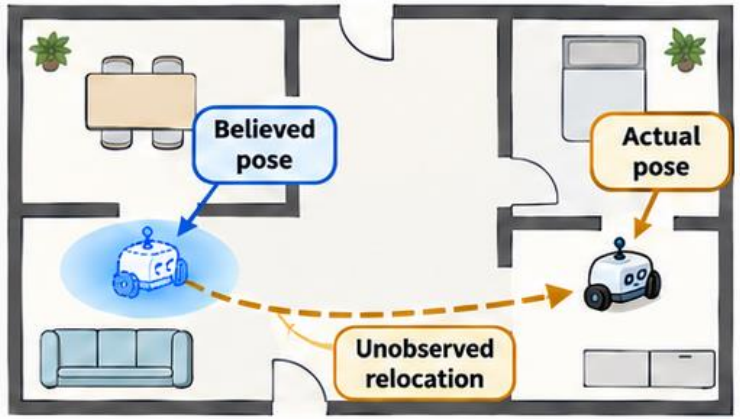
Unknown initial pose

Goal: Find the robot anywhere in the map.

Challenge: Resolve multiple plausible locations.

Example: Switching on at an unknown location.

3 Kidnapped robot



Confident, but wrong pose

Goal: Detect failure and localize again.

Why kidnapped? Moved without the estimator knowing.

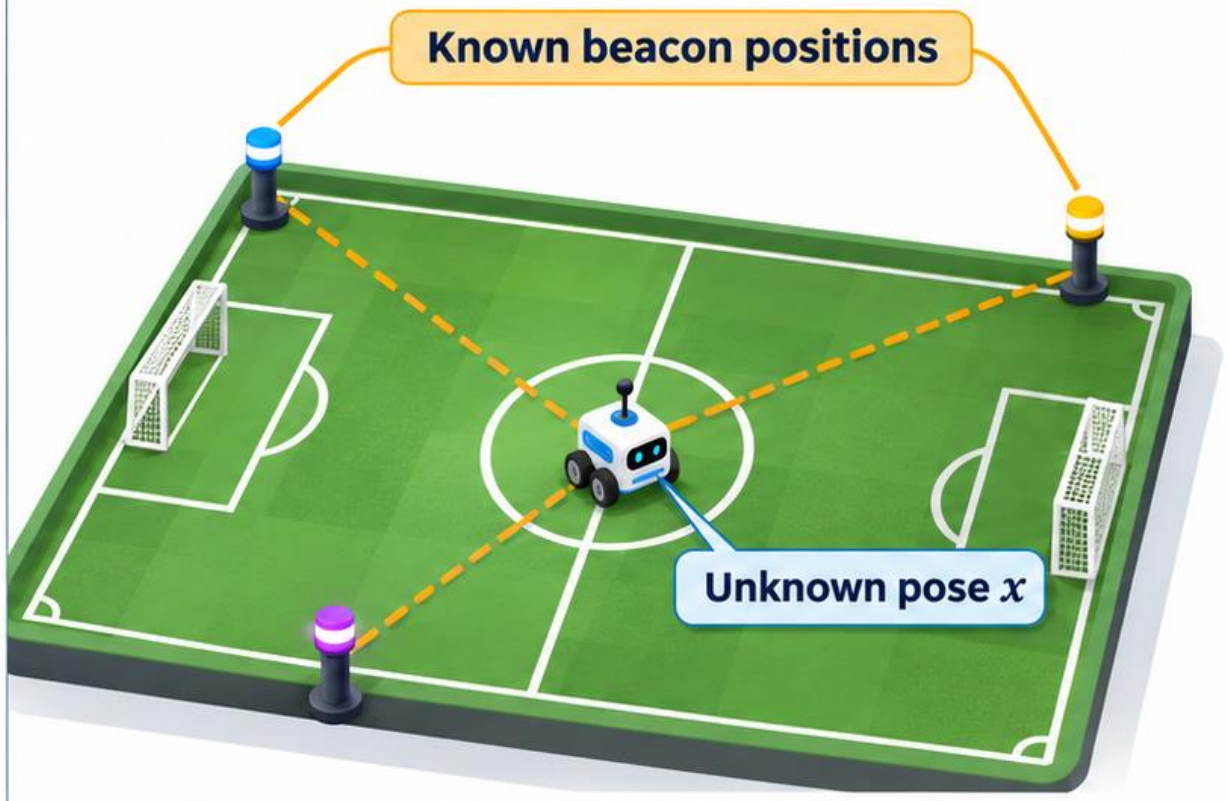
Example: Picked up and placed elsewhere.

Pose = position + orientation Blue markers = pose hypotheses

Track a known pose • Find an unknown pose • Recover from a wrong pose

Least Squares Positioning

Known landmarks, unknown pose



Measurements provide constraints on the pose.

1 Measurement model

$$\mathbf{z} = \mathbf{h}(\mathbf{x}) + \mathbf{e}$$

- $\mathbf{z}$ measured data
- $\mathbf{x}$ unknown state
- $\mathbf{h}$ nonlinear observation function
- $\mathbf{e}$ measurement noise

2 Best fit

$$\hat{\mathbf{x}} = \arg\min_{\mathbf{x}} \|\mathbf{z} - \mathbf{h}(\mathbf{x})\|^2$$

Choose the state that minimizes squared residuals.

Residual = measured – predicted

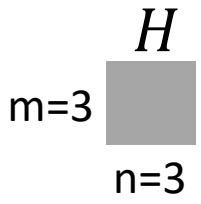
Noisy measurements rarely agree exactly. Least squares finds the best compromise.

Least Squares Positioning (linear case):

Given $\mathbf{z} = H\mathbf{x} + \mathbf{e}$ find the best pose $\hat{\mathbf{x}}$

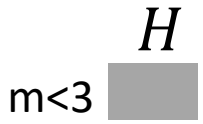
H is a matrix of dimension mxn

- n: dimension of the state (3 for planar poses)
- m: #observation constraints (equations)



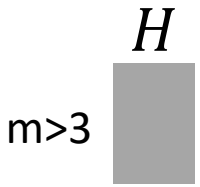
m=3: if the 3 equations are independent -> $\text{rank}(H) = 3 \rightarrow H_{\mathbf{3x3}}$ is invertible

- direct solution for $\mathbf{x}$: $\hat{\mathbf{x}} = H^{-1}\mathbf{z}$



m<3: $\text{rank}(H) < \text{n} = 3 \rightarrow (H^T H)_{3\text{x}3}$ not invertible

- The pose is Non-observable: The pose $\mathbf{x}$ cannot be uniquely determined



m>3: if at least 3 independent constraints -> $\text{rank}(H) = 3 \rightarrow (H^T H)_{3\text{x}3}$ invertible

- Generally, no exact solution because of measurement noise
- Find $\mathbf{x}$ that $H\mathbf{x}$ is as close as possible to $\mathbf{z}$ → minimum squared error

Cost function $\|\mathbf{e}\|^2$ — Cost function

$$\hat{\mathbf{x}} = \arg\min_{\mathbf{x}} \mathbf{e}^T\mathbf{e} = \arg\min_{\mathbf{x}} [(\mathbf{z} - H\mathbf{x})^T(\mathbf{z} - H\mathbf{x})] = \arg\min_{\mathbf{x}} \|\mathbf{z} - H\mathbf{x}\|^2$$

Solution: $\hat{\mathbf{x}} = (H^T H)^{-1} H^T \mathbf{z}$ Least-squares estimate

Weighted Least Squares

- **Measurements are not equally reliable**: different "importance" of the noise error e_i of each measurement: $e_i = z_i - [H\boldsymbol{x}]_i$ Element i of the vector $H\mathbf{x}$

- If measurement errors e_i are mutually independent, Q is diagonal.

$$Q = \begin{bmatrix} \sigma_{z_1}^2 & 0 & \cdots \\ 0 & \sigma_{z_2}^2 & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix} \quad Q^{-1} = \begin{bmatrix} \frac{1}{\sigma_{z_1}^2} & 0 & \cdots \\ 0 & \frac{1}{\sigma_{z_2}^2} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}$$

- Each element e_i of the error vector $\mathbf{e}$ is inversely weighted by the uncertainty (variance) of measurement z_i: $1/\sigma_{z_i}^2$

Cost function

$$\hat{\mathbf{x}} = \arg\min_{\mathbf{x}} \mathbf{e}^T Q^{-1} \mathbf{e} = \arg\min_{\mathbf{x}} [(\mathbf{z} - H\mathbf{x})^T Q^{-1} (\mathbf{z} - H\mathbf{x})] = \arg\min_{\mathbf{x}} \sum_{i=1}^{m} \frac{e_i^2}{\sigma_{z_i}^2}$$

Solution:

[Closed form]

$$\hat{\mathbf{x}} = (H^T Q^{-1} H)^{-1} H^T Q^{-1} \mathbf{z}$$

Best estimation

$$\Sigma_{\hat{\mathbf{x}}} = (H^T Q^{-1} H)^{-1}$$

Covariance of the estimation (propagating from Q)

Weighted Least Squares (WLS) equivalent to ML Estimation if error is Gaussian

Maximum Likelihood (ML) Estimation

Find $\mathbf{x}$ that maximizes the likelihood function $\mathcal{L}(\mathbf{x}) = p(\mathbf{z}|\mathbf{x})$

$$\mathbf{x}_{ML} = \arg\max_{\mathbf{x}} p(\mathbf{z}|\mathbf{x})$$

If the error in the measurements is zero-mean Gaussian, i.e. $\mathbf{e} \sim N(0, Q)$

$$\mathbf{z} = H\mathbf{x} + \mathbf{e} \Rightarrow \mathbf{z} \sim N(H\mathbf{x}, Q)$$

$$p(\mathbf{z}|\mathbf{x}) = K \cdot \exp\{-\frac{1}{2}[\mathbf{z} - H\mathbf{x}]^T Q^{-1}[\mathbf{z} - H\mathbf{x}]\}$$

$$\hat{\mathbf{x}}_{ML} = \arg\max_{\mathbf{x}} p(\mathbf{z}|\mathbf{x}) = \arg\max_{\mathbf{x}} \{-[\mathbf{z} - H\mathbf{x})]^T Q^{-1}[\mathbf{z} - H\mathbf{x}]\}$$

$$= \arg\min_{\mathbf{x}} [(\mathbf{z} - H\mathbf{x})^T Q^{-1}(\mathbf{z} - H\mathbf{x})] = \underbrace{\arg\min_{\mathbf{x}} \mathbf{e}^T Q^{-1}\mathbf{e}}_{\text{Weighted Least Squares}} = \hat{\mathbf{x}}_{LS}$$

The LS and ML solutions are the same $\hat{\mathbf{x}}_{ML} = \hat{\mathbf{x}}_{LS}$

Non-linear Least Squares

Non-linear observation (sensor) model: $\mathbf{z} = \mathbf{h}(\mathbf{x})$ (instead of $\mathbf{z} = H\mathbf{x}$)

$$\hat{\mathbf{x}} = \arg\min_{\mathbf{x}} \|\mathbf{z} - \mathbf{h}(\mathbf{x})\|^2$$

These is a vector of m numbers (observations), not RVs!

Unlike linear LS, this generally has **no direct closed-form solution**, so we start from an estimate $\mathbf{x}_k$ and locally linearize (with Taylor expansion):

$$\mathbf{h}(\mathbf{x}) = \mathbf{h}(\mathbf{x}_k + \boldsymbol{\delta}) \cong \mathbf{h}(\mathbf{x}_k) + J_{h_k}\boldsymbol{\delta}$$

Jacobian of $\mathbf{h}$ evaluated at $\mathbf{x}_k$

$$J_{h_k} = \frac{d\mathbf{h}}{d\mathbf{x}}\bigg|_{\mathbf{x}_k} = \begin{bmatrix} \frac{dh_1}{dx_1} & \cdots & \frac{dh_1}{dx_n} \\ \vdots & & \vdots \\ \frac{dh_m}{dx_1} & \cdots & \frac{dh_m}{dx_n} \end{bmatrix}_{\mathbf{x}_k}$$

mxn

This converts each iteration into a **linear LS problem in $\boldsymbol{\delta}$**:

$$\boldsymbol{\delta}^* = \arg\min_{\boldsymbol{\delta}} \left\|\mathbf{z} - \mathbf{h}(\mathbf{x}_k) - J_{h_k}\boldsymbol{\delta}\right\|^2 \Longrightarrow \mathbf{x}_{k+1} = \mathbf{x}_k + \boldsymbol{\delta}^*$$

Repeat until $\boldsymbol{\delta}^*$ is sufficiently small

Weighted Non-linear Least Squares

Non-Weighted: $\boldsymbol{\delta} = \left(J_{\mathbf{h}}^T J_{\mathbf{h}}\right)^{-1} J_{\mathbf{h}}^T \mathbf{e}$

Weighted: $\boldsymbol{\delta} = \left(J_{\mathbf{h}}^T Q^{-1} J_h\right)^{-1} J_{\mathbf{h}}^T Q^{-1} \mathbf{e}$

Weighting matrix

Algorithm **Gauss–Newton for nonlinear weighted LS**

Begin with an initial guess $\hat{\mathbf{x}}$

1. Compute the incremental pose $\boldsymbol{\delta} = \left[\boldsymbol{\delta}_{xy}^T, \delta_\theta\right]^T$

$$\boldsymbol{\delta} = \left(J_{\mathbf{h}}^{\boldsymbol{T}} Q^{-\mathbf{1}} J_{\mathbf{h}}\right)^{-\mathbf{1}} J_{\mathbf{h}}^{\boldsymbol{T}} Q^{-\mathbf{1}} [\mathbf{z} - \mathbf{h}(\hat{\mathbf{x}})]$$

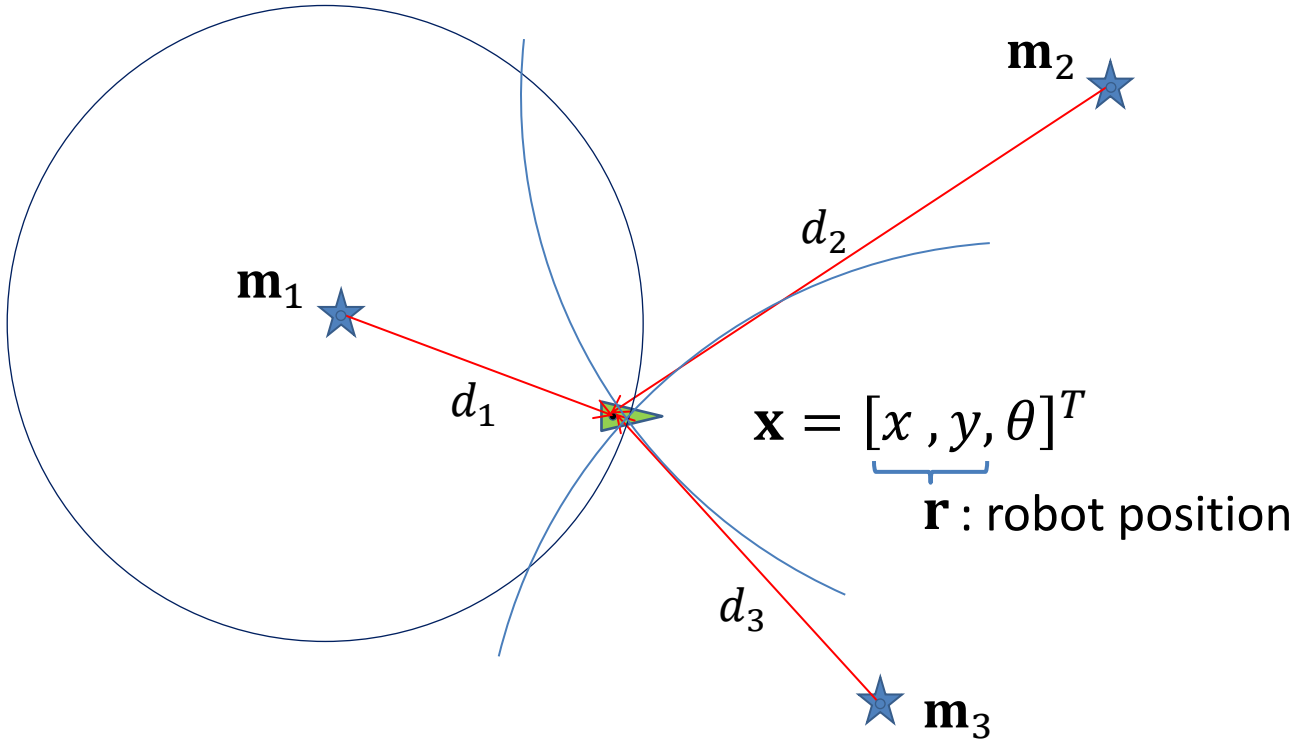
2. Set $\hat{\mathbf{x}} = \hat{\mathbf{x}} + \boldsymbol{\delta}$
3. If $\left\|\boldsymbol{\delta}_{xy}\right\| < \mathrm{T}_1$, $|\delta_\theta| < T_\theta$ stop, else goto 1 with $\hat{\mathbf{x}}$

Least Squares Positioning

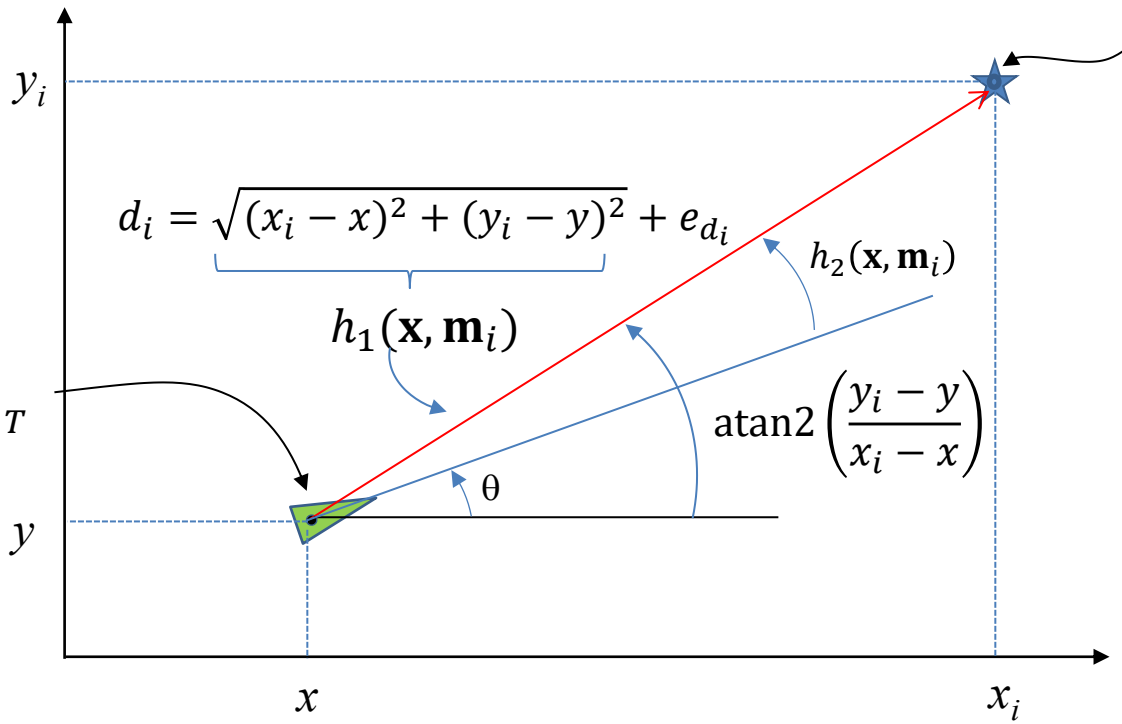
Example: Multilateration(also called *Range-only positioning*) in 2D

Estimate the position $\mathbf{r} = [x\ , y]^T$ of a vehicle in 2D, given distances d_i to *n* beacons $\mathbf{m}_i$ (sort of indoor GPS)

$\mathbf{m}_i = [x_i\ , y_i\ ]^T$: position of beacon i (given)



The range-bearing observation model (revisited)



Known position of the landmark $\mathbf{m}_i$

Sensor error: $\mathbf{e}_i = \begin{bmatrix} e_{d_i} \\ e_{\theta_i} \end{bmatrix}$

True robot pose (unknown)

$$\mathbf{x} = \underbrace{[x, y, \theta]^T}_{\mathbf{r}} = [\mathbf{r}, \theta]^T$$

Observation model

$$\mathbf{z}_i = \begin{bmatrix} d_i \\ \theta_i \end{bmatrix} = \underbrace{\mathbf{h}(\mathbf{x}, \mathbf{m}_i)}_{\text{Predicted observation}} + \mathbf{e}_i = \begin{bmatrix} \sqrt{(x_i - x)^2 + (y_i - y)^2} \\ \text{atan}(\frac{y_i - y}{x_i - x}) - \theta \end{bmatrix} + \mathbf{e}_i$$

Jacobian (wrt the robot pose $\mathbf{x}$)

$$J_{\mathbf{h},\mathbf{x}} = \frac{\partial \mathbf{h}}{\partial \{x, y, \theta\}} = \begin{bmatrix} -\frac{x_i - x}{\|\mathbf{m_i} - \mathbf{r}\|} & -\frac{y_i - y}{\|\mathbf{m_i} - \mathbf{r}\|} & 0 \\ \frac{y_i - y}{\|\mathbf{m_i} - \mathbf{r}\|^2} & -\frac{x_i - x}{\|\mathbf{m_i} - \mathbf{r}\|^2} & -1 \end{bmatrix}_{2x3}$$

In this example of trilateration we only need the range component h_1

Example 3 beacons: Trilateration (Range-only positioning) in 2D

General observation model: $\mathbf{z} = \mathbf{h}(\mathbf{x}) + \mathbf{e}$

For a range sensor: $\mathbf{h}(\mathbf{x}) = \mathbf{h}(\mathbf{r}) = \begin{bmatrix} h_1(\mathbf{r}) \\ h_2(\mathbf{r}) \\ h_3(\mathbf{r}) \end{bmatrix} = \begin{bmatrix} \|\mathbf{m}_1 - \mathbf{r}\| \\ \|\mathbf{m}_2 - \mathbf{r}\| \\ \|\mathbf{m}_3 - \mathbf{r}\| \end{bmatrix} = \begin{bmatrix} [(x_1 - x)^2 + (y_1 - y)^2]^{1/2} \\ [(x_2 - x)^2 + (y_2 - y)^2]^{1/2} \\ [(x_3 - x)^2 + (y_3 - y)^2]^{1/2} \end{bmatrix}$

Distance of $\mathbf{r}$ (unknown) to each landmark $\mathbf{m}_i$ (known)

Do not depend on the robot heading θ ($\frac{\partial h_i}{\partial \theta} = 0$)

θ is not observable from range−only measurements

$$\mathbf{e} = \mathbf{z} - \mathbf{h}(\mathbf{r}) = \begin{bmatrix} d_1 - h_1(\mathbf{r}) \\ d_2 - h_2(\mathbf{r}) \\ d_3 - h_3(\mathbf{r}) \end{bmatrix} = \begin{bmatrix} e_1 \\ e_2 \\ e_3 \end{bmatrix}$$

Derivatives

$$\frac{\partial h_1}{\partial x} = \frac{\partial}{\partial x}[(x_i - x)^2 + (y_i - y)^2]^{1/2} = -\frac{1}{\|\mathbf{m_i} - \mathbf{r}\|}(x_i - x)$$

$$\frac{\partial h_1}{\partial y} = \frac{\partial}{\partial y}[(x_i - x)^2 + (y_i - y)^2]^{1/2} = -\frac{1}{\|\mathbf{m_i} - \mathbf{r}\|}(y_i - y)$$

Jacobian of $\mathbf{h}$

$$J_{\mathbf{h}} = \begin{bmatrix} -\frac{1}{\|\mathbf{m_1} - \mathbf{r}\|}(x_1 - x) & -\frac{1}{\|\mathbf{m_1} - \mathbf{r}\|}(y_1 - y) \\ -\frac{1}{\|\mathbf{m_2} - \mathbf{r}\|}(x_2 - x) & -\frac{1}{\|\mathbf{m_2} - \mathbf{r}\|}(y_2 - y) \\ -\frac{1}{\|\mathbf{m_3} - \mathbf{r}\|}(x_3 - x) & -\frac{1}{\|\mathbf{m_3} - \mathbf{r}\|}(y_3 - y) \end{bmatrix}_{\mathbf{3x2}}$$

Least square problem: $\hat{\mathbf{r}} = [\hat{x}, \hat{y}]^{\mathrm{T}} = \arg\min_{\mathbf{r}} \mathbf{e}^T\mathbf{e} = \arg\min_{\mathbf{r}} [e_1^2 + e_2^2 + e_3^2]$

Solution: Iteratively solve $\boldsymbol{\delta}_{2x1} = \left(J_{\mathbf{h}}^T J_{\mathbf{h}}\right)_{2x2}^{-1} J_{\mathbf{h}\,2x3}^T [\mathbf{z} - \mathbf{h}(\hat{\mathbf{r}})]_{3x1}$

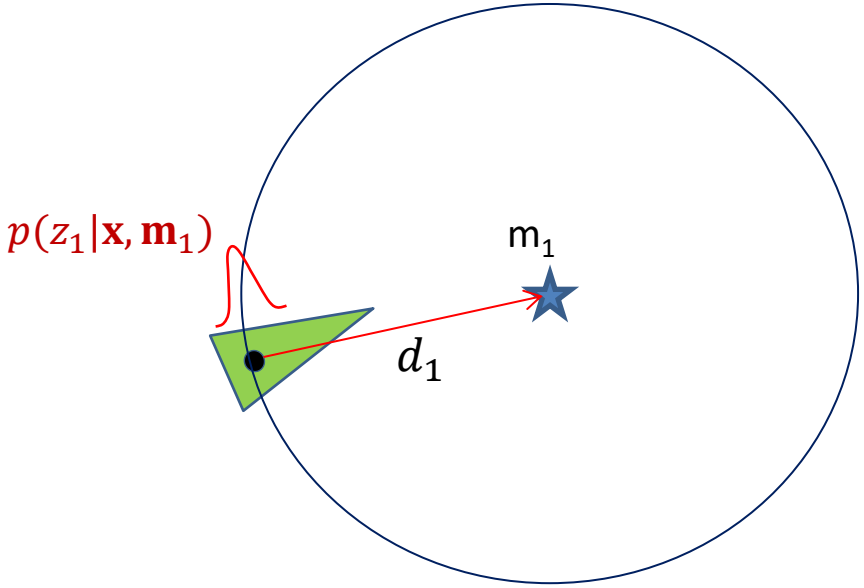
Example: Range-only positioning in 2D (cont.)

If just one landmark is observed, estimating $\mathbf{r} = [x, y]^T$ has infinitely many solutions!

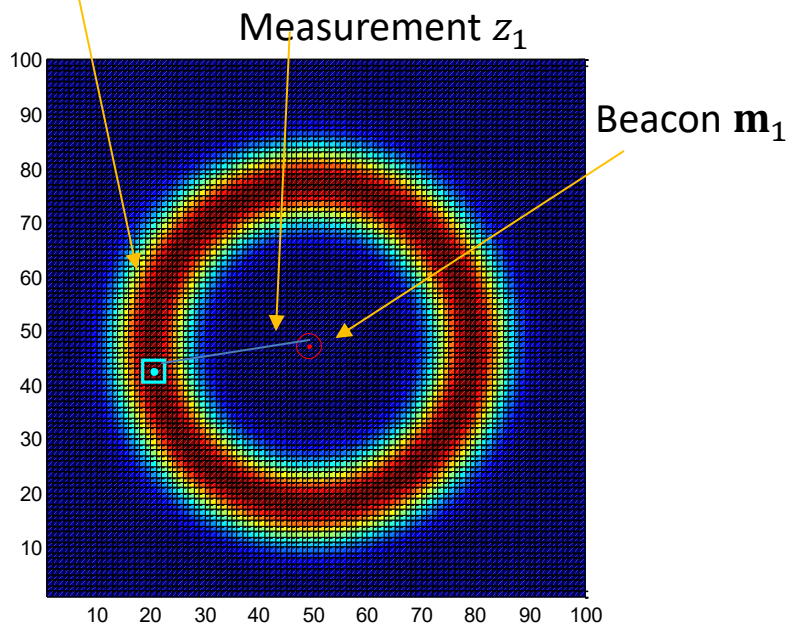
$$z_1 = d_1 = [(x_1 - x)^2 + (y_1 - y)^2]^{1/2} + e_1$$

The measurement z_1 tells us that the robot is on a circumference with radius d_1

With $e_1 \sim N(0, \sigma^2)$, the robot position uncertainty will be given by a "gaussian donut" → **It is not a Gaussian distribution**



Likelihood $\mathcal{L}(x) = p(z_1|\mathbf{x}, \mathbf{m}_1)$ (z_1 and $\mathbf{m}_1$ given)



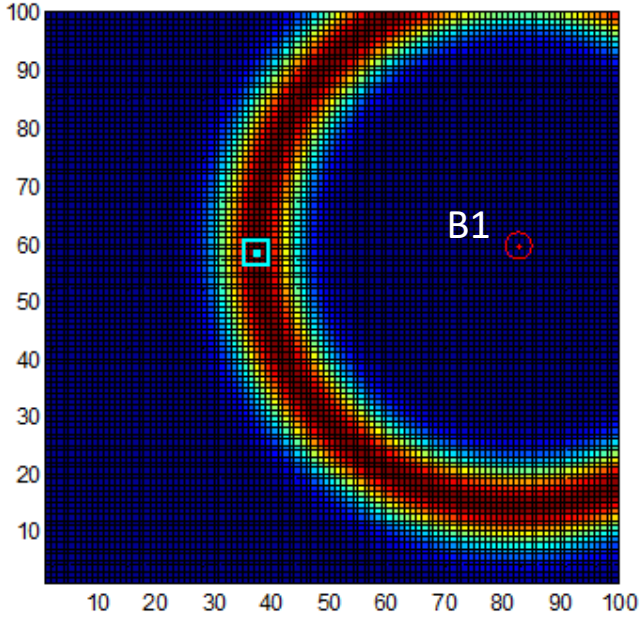
Bayes: $p(\mathbf{x}|z_1, \mathbf{m}_1) = kp(z_1|\mathbf{x}, \mathbf{m}_1)p(\mathbf{x}|\mathbf{m}_1)$

$if\ p(\mathbf{x}|\mathbf{m}_1)\ \ uniform \rightarrow p(\mathbf{x}|z_1, \mathbf{m}_1) = k'p(z_1|\mathbf{x}, \mathbf{m}_1)$

Notice: $p(z|\mathbf{x}, \mathbf{m}_1)$ is Gaussian BUT $\mathcal{L}(\mathbf{x}) = p(z_1|\mathbf{x}, \mathbf{m}_1)$ IS NOT (it has "donut shape")

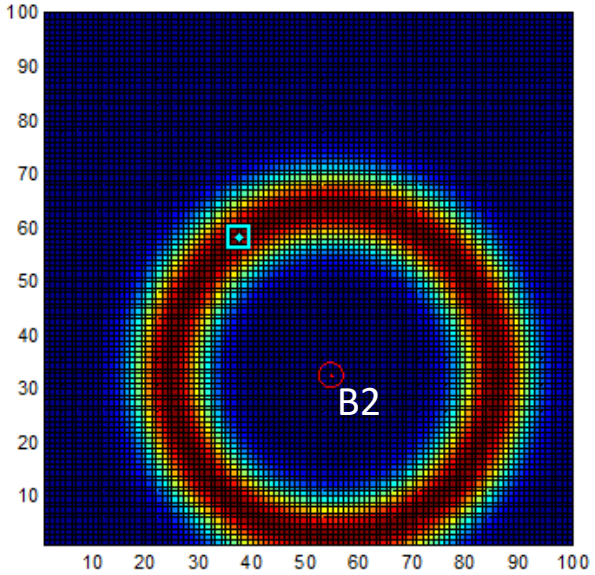
Two observed ranges combined

Observation 1:
distance to beacon B1



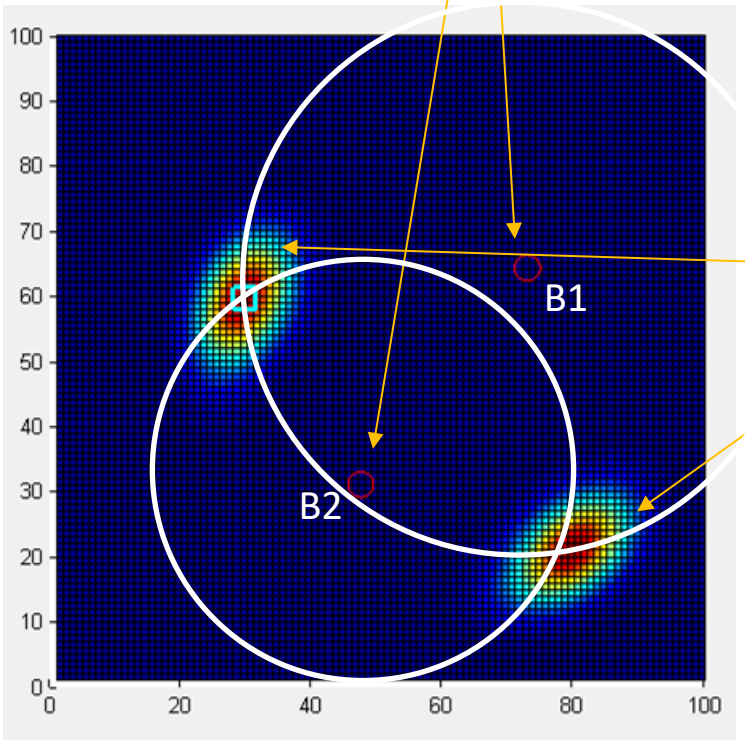
$p(z_1|x, \mathbf{m}_1)$

Observation 2:
distance to beacon B2



$p(z_2|\mathbf{x}, \mathbf{m}_2)$

Beacons $M = \{\mathbf{m}_1, \mathbf{m}_2\}$ given



Uncertainty in the robot pose:
$p(\mathbf{x}|\, z_1, z_2, M)$

Product of the two donut-circles
$p(z_1|\mathbf{x}, M)\; p(z_2|\mathbf{x}, M)$

$if p(\mathbf{x}|M)\;\; (prior)\; uniform$

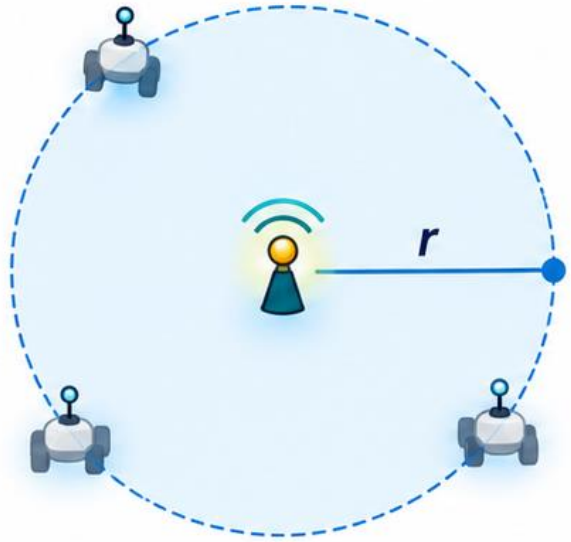
$$p(\mathbf{x}|\, z_1, z_2, M) = k' p(z_1, z_2|\mathbf{x}, M) =$$
$$= k' p(z_1|\mathbf{x}, M)\; p(z_2|\mathbf{x}, M)$$

Conditional independence

Least Squares Positioning

Characteristics

Enough information

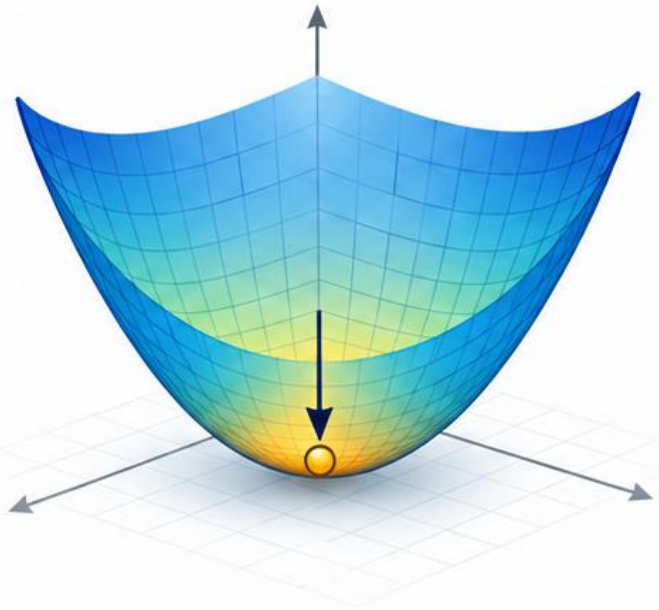


One range → many positions

Unique pose needs sufficient constraints

A prior can constrain the estimate

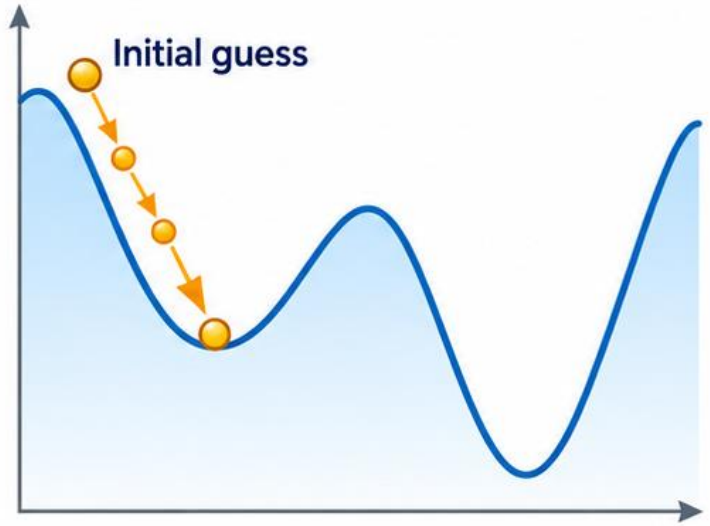
Linear LS



Direct solution

Global minimum

Nonlinear LS



Iterative solution

May reach a local minimum



Gaussian measurement noise: weighted LS = maximum likelihood

Ordinary LS assumes equal, independent noise variance.

Point-set registration

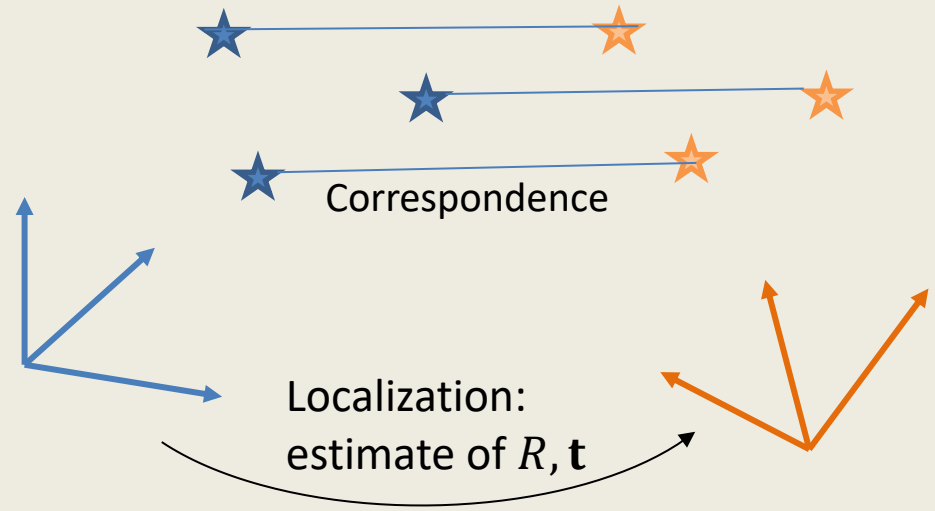
A set of observed points are **registered (overlapped) against …**

- A global map → gives us the global pose of the observation (**global localization**)
- A previously observed point set → gives us their relative pose (**visual odometry**)

Two types of points:

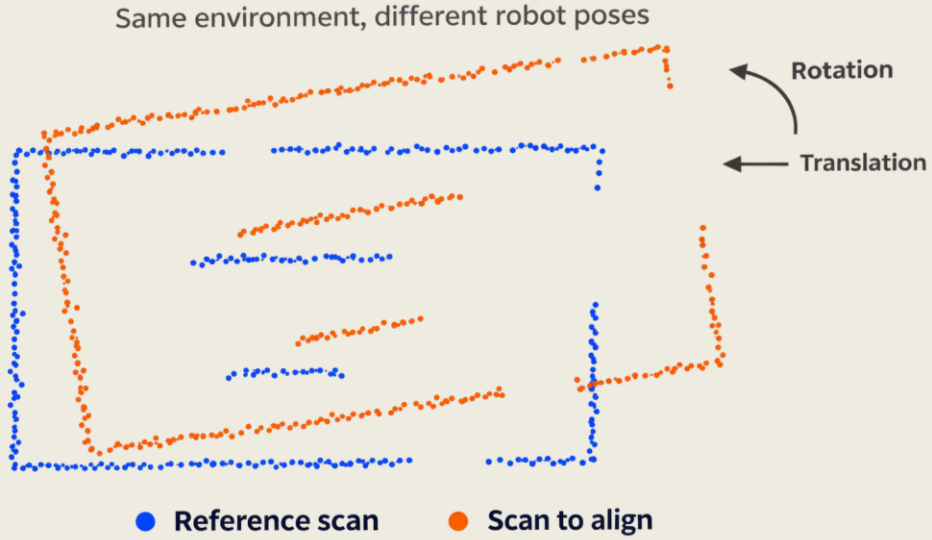
- Landmark points:

Correspondences are typically known (from landmarks descriptors)



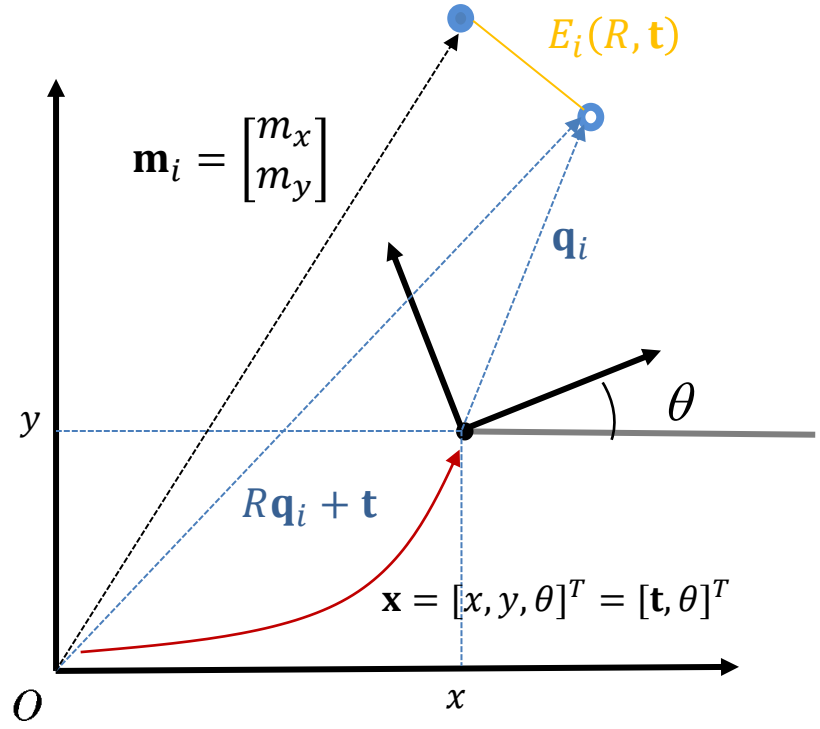
- Scan points:

Point correspondences are unknown (points do not have a descriptor for matching)



For **landmark points:** known correspondences assumed

Consider two sets of points $\{\mathbf{m}_i\}_{i=1}^N$ and $\{\mathbf{q}_i\}_{i=1}^N$, with pairs $(\mathbf{m}_i, \mathbf{q}_i)$ in correspondence



$$\mathbf{x} \oplus \mathbf{q} = \begin{bmatrix} x + q_x \cos\theta - q_y \sin\theta \\ y + q_x \sin\theta + q_y \cos\theta \end{bmatrix} = \mathbf{t} + R\mathbf{q}$$

Error for one point $\mathbf{q}_i$:

$$E_i(R,\mathbf{t}) = \|\mathbf{m}_i - \mathbf{x} \oplus \mathbf{q}_i\|^2 = \|\mathbf{m}_i - (R\mathbf{q}_i + \mathbf{t})\|^2$$

Optimization for the whole error $E(R,\mathbf{t})$:

$$\{\hat{\mathbf{t}}, \hat{R}\} = \arg\min_{\mathbf{t},R} \sum_{i=1}^{n} \|\mathbf{m}_i - (R\mathbf{q}_i + \mathbf{t})\|^2$$

Find the translation $\mathbf{t}$ and rotation R that minimize the sum of the squared error $E(R,\mathbf{t})$

$R, \mathbf{t}$ can be calculated iteratively (e.g. Gauss-Newton) or in closed-form (via SVD)

Scan Matching: The Iterated Closest Point (**ICP**) Algorithm

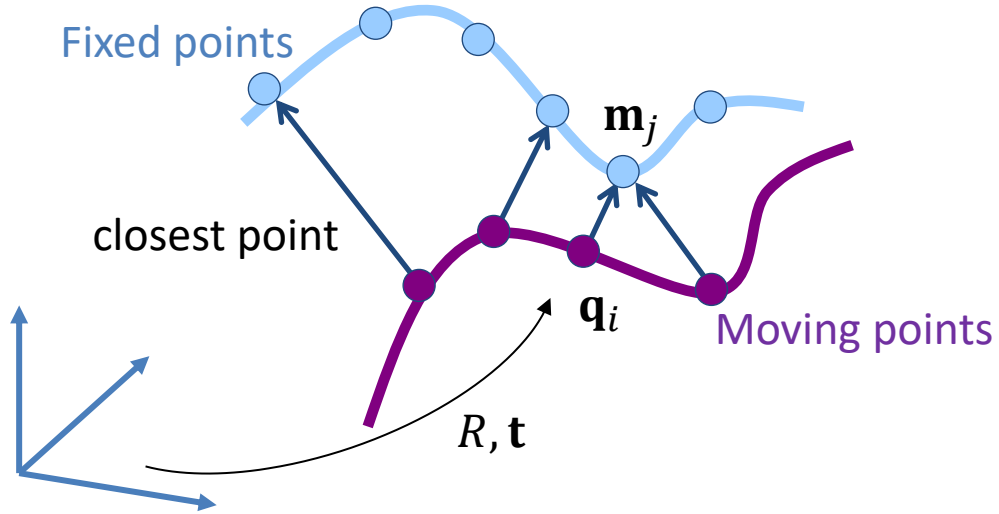
Points do not have a descriptor for matching

ICP Algorithm: Iterate until convergence:

1. For each $\mathbf{q}_i$ in one set, make a pair with the **closest point** $\mathbf{m}_j$ in the other set
2. Solve one step in the Gauss-Newton method for the minimization of:

$$E(R,\mathbf{t}) = \sum_{i=1}^{n} \left\| \mathbf{m}_j - R\mathbf{q}_i - \mathbf{t} \right\|^2$$

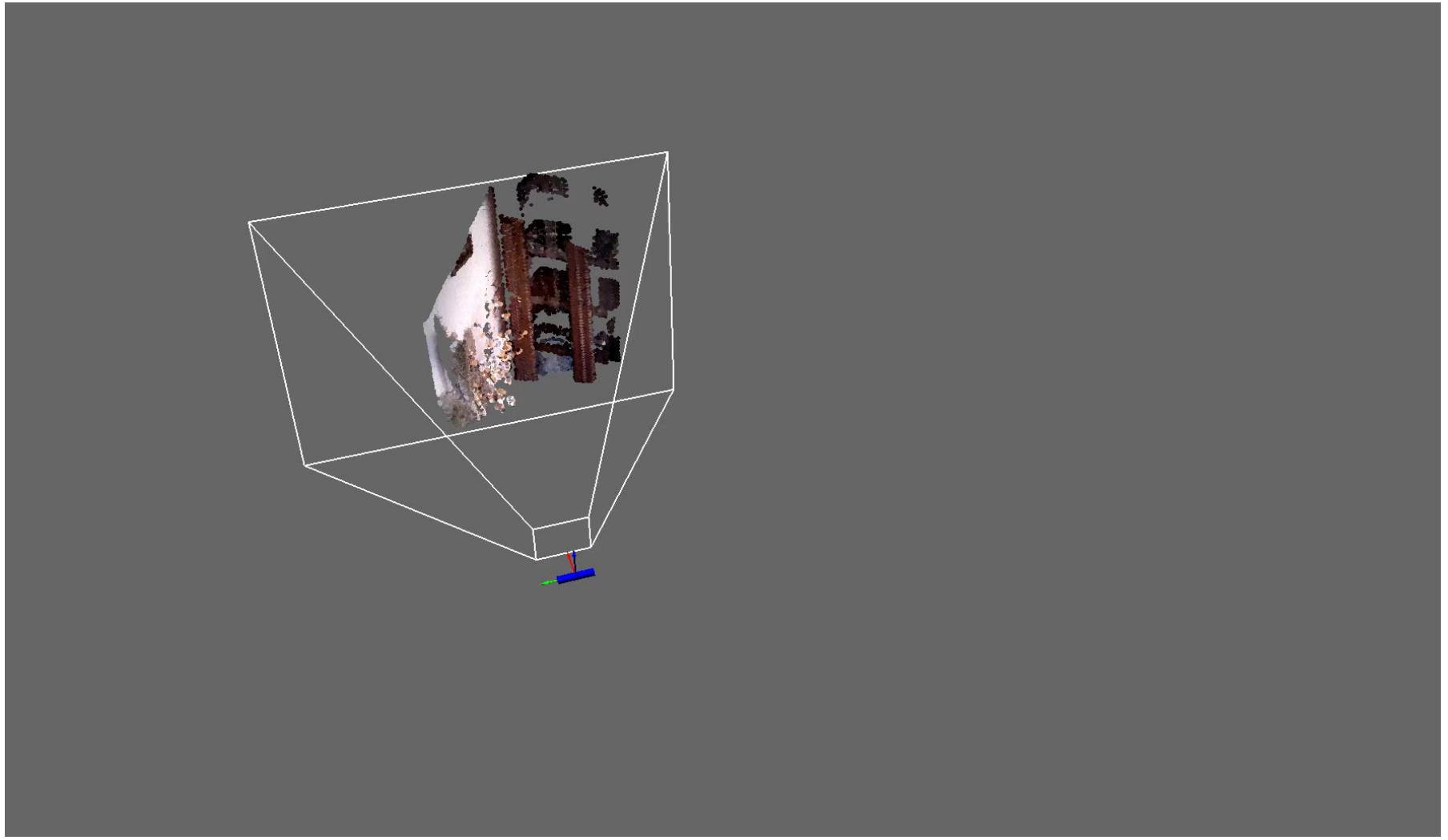
Convergence: computed $R,\mathbf{t}$ in step 2 are negligible or the pairs in step 1 do not change



- ICP is **guaranteed to converge** to a **local minimum** only, not a global one
- Closest-point matching is generally stable, but **slow** and requires preprocessing (KD-tree technique)

ICP: Example

Registration of 3D points from a sequence of RGB-D images

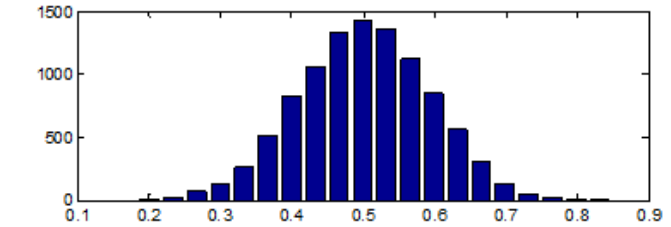
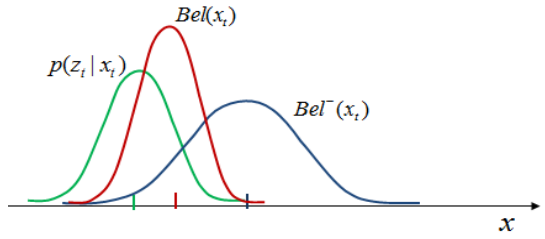
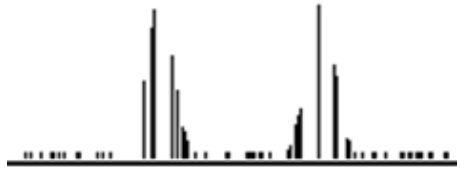


[Jaimez & Gonzalez-Jimenez, 2015]

Bayes Filtering for Probabilistic Localization

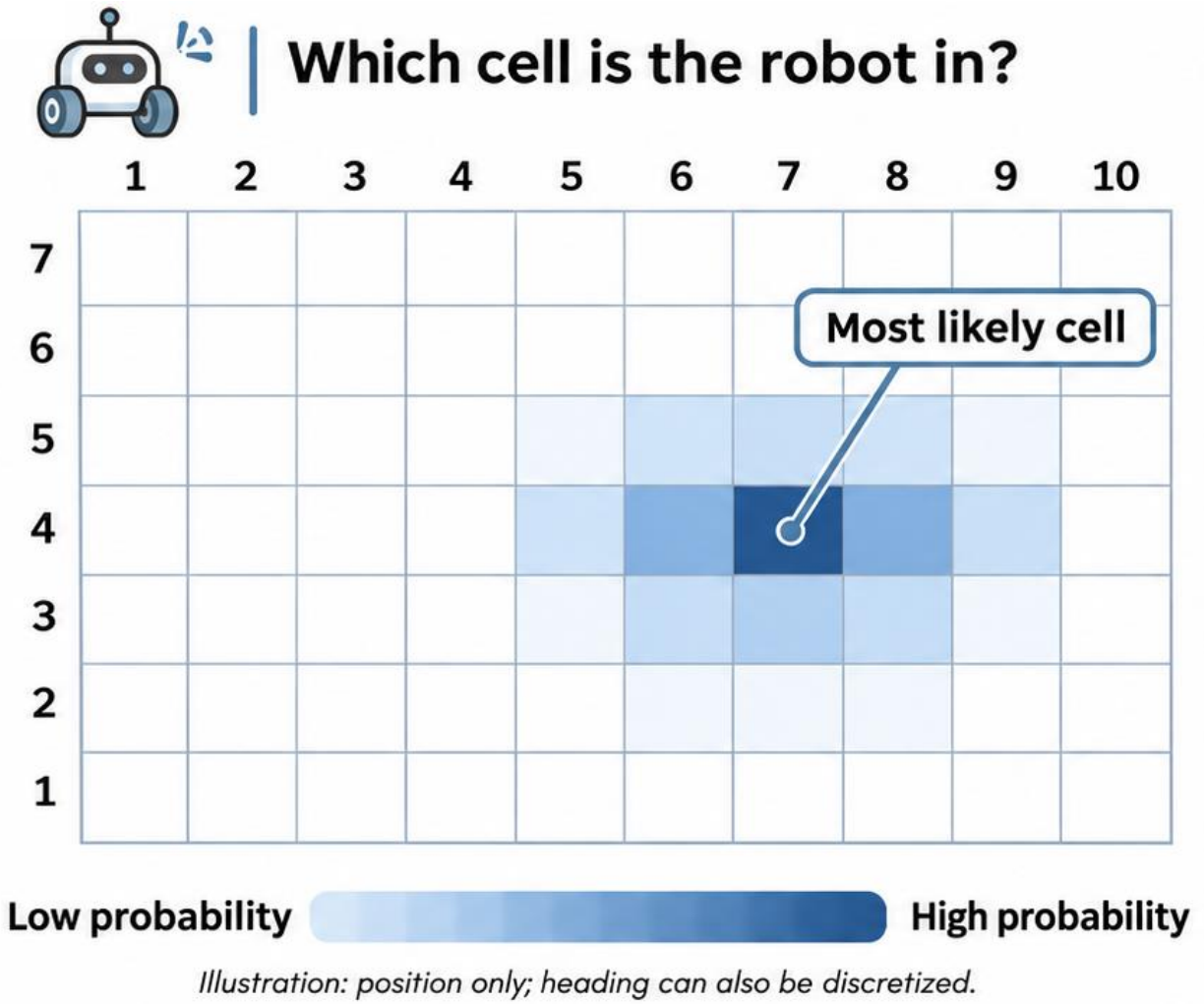
From a single pose estimate $\hat{\mathbf{x}}$ to a **belief over possible poses:** $p(\mathbf{x}_t | \mathbf{z}_{0:t}, \mathbf{u}_{0:t})$

Bayes Filtering → Repeat: Predict from motion → Update with measurements

Method	Belief representation	Variable x_t	Example 1 D
Grid/histogram Filter	Probability per cell	Discrete	
Kalman Filter	Gaussian $(\boldsymbol{\mu}, \Sigma)$	Continuous	
Particle Filter	Weighted samples	Continuous	

Same Bayes principle—different representations of uncertainty.

Grid Filter: Localization in a grid



A probability for every cell

$$p_i = \mathbf{P}(\mathbf{X} = \text{cell}_i)$$

$$\sum_i p_i = 1$$

The grid represents the robot's belief.

1 Predict — motion

Move and spread the belief.



2 Update — sensing

Favor cells consistent with measurements.



The probability over the cells is represented by a histogram.

Example: *Belief* of a robot position over a grid

$P(r)$

0.01
0.06
0.42
0.24
0.19
0.08

Joint probability distribution $P(row, column)$

			0.01				
		0.01	0.01	0.02	0.02		
		0.02	0.1	0.15	0.1	0.05	
		0.02	0.05	0.1	0.05	0.02	
		0.01	0.01	0.1	0.05	0.02	
				0.01	0.05	0.02	

$r=3$

r

c

$c=5$

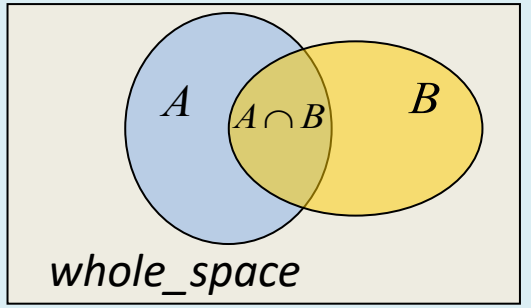
$P(c)$

		0.06	0.18	0.38	0.27	0.11	

Given the joint distribution we can answer questions like:

What is the probability that the robot is in row 3 (*A*) OR in column 5 (*B*):

A, B are outcomes with the same whole space



Whole space: all the posible cells (8x8)

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

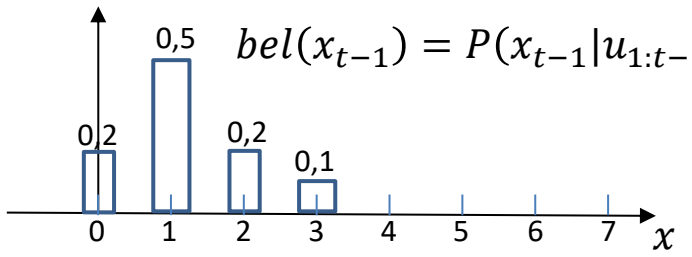
$$A: row\ r = 3$$

$$B: column\ c = 5$$

$$P(A \cup B) = P(r = 3) + P(c = 5) - P(r = 3, c = 5)$$
$$= 0.19 + 0.38 - 0.1 = 0.47$$

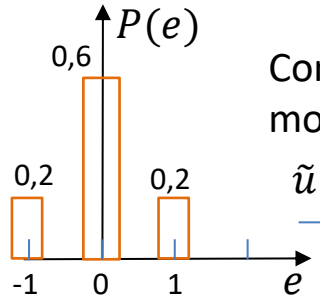
Grid Bayes Filter. Example in 1D

Robot in a **tiled corridor (1D –Grid)** with the following starting belief over the position $x \in \{0,1,\ldots,7\}$



$bel(x_{t-1}) = P(x_{t-1}|u_{1:t-1}, z_{1:t-1})$= [0.2, 0.5, 0.2., 0.1, 0, 0, 0, 0]

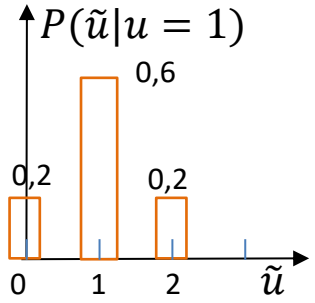
Motion error



Commanded motion $u = 1$ cell

$\tilde{u} = 1 + e$

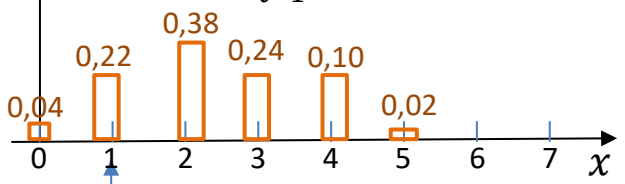
Actual motion distribution for $u = 1$



The robot may stay (0.2), move one cell (0.6), or move two cells (0.2).

***Predicted* pose distribution**

$$\overline{bel}(x_t) = \sum_{x_{t-1}} P(x_t \mid x_{t-1}, u_t)\, bel(x_{t-1})$$



Motion typically spreads uncertainty

Which previous positions can bring probability to this cell?

$x_t = x_{t-1} + \tilde{u}_t,$

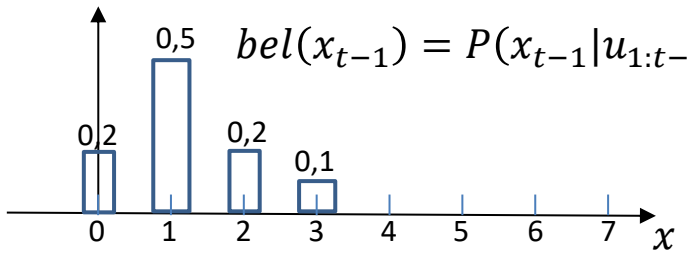
One example: $\overline{bel}_t(1) = 0.6 * bel(0) + 0.2 * bel(1) = 0.22$

is at cell 0 (0.2), moves 1 cell (0.6)

is at cell 1 (0.5), moves 0 cell (0.2)

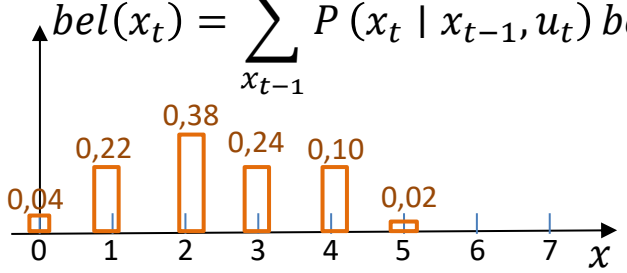
Grid Bayes Filter. Example in 1D

Robot in a **tiled corridor (1D –Grid)** with the following starting Belief over the position $x \in \{0,1,\ldots,7\}$



***Predicted* pose distribution**

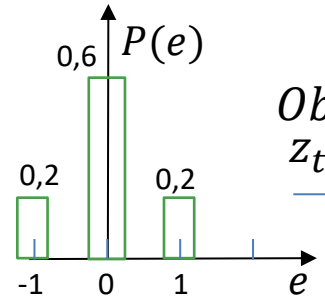
$$\overline{bel}(x_t) = \sum_{x_{t-1}} P(x_t \mid x_{t-1}, u_t)\, bel(x_{t-1})$$



Motion typically spreads uncertainty

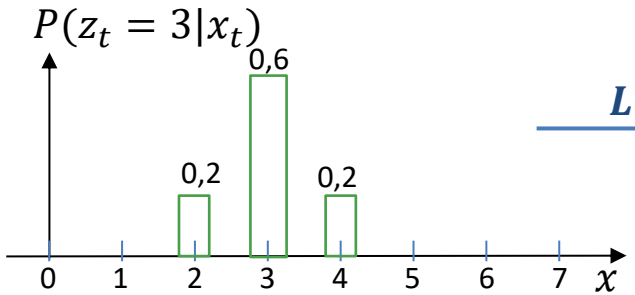
Prior

Sensor error



Observation $z_t = x_t + e_t$

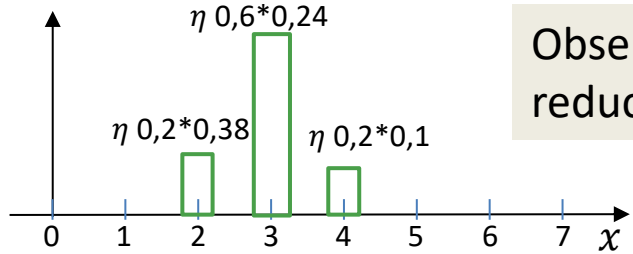
Observation *likelihood* for $z = 3$



Likelihood

Correction (Apply Bayes)* → *Posterior

$$bel(x_t) = P(x_t|u_{1:t}, z_{1:t}) = \eta P(z_t|x_t)\overline{bel}(x_t)$$



Observation typically reduces uncertainty

$$\eta = \frac{1}{0.24},$$

Grid Bayes Filter. Conceptual idea

$\underbrace{bel(x_{t-1})}$ $\xrightarrow{p(x_t|x_{t-1},u_t)}$ $\underbrace{\overline{bel}(x_t)}$ $\xrightarrow{p(z_t|x_t)}$ $\underbrace{bel(x_t)}$

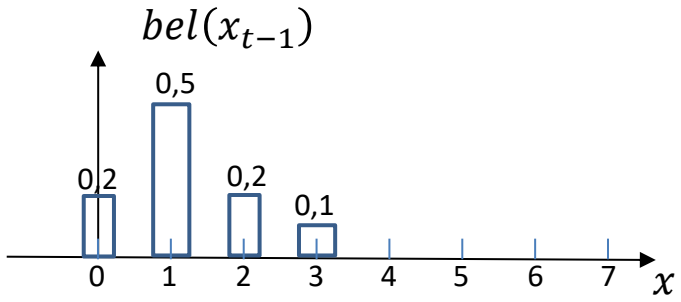
previous posterior — predicted belief / prior — new posterior

Robot Motion

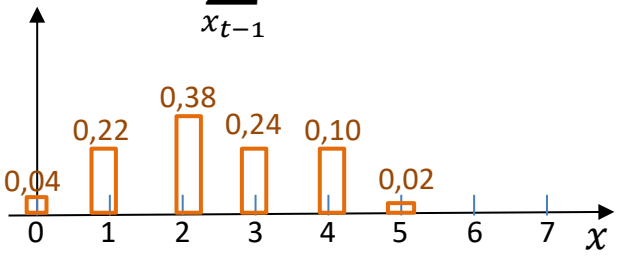
Total probability

Observation

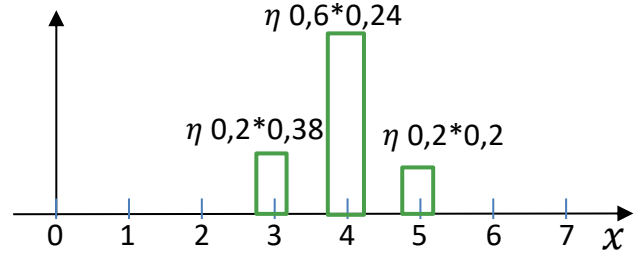
Bayes



$$\overline{bel}(x_t) = \sum_{x_{t-1}} P\,(x_t \mid x_{t-1}, u_t)\, bel(x_{t-1})$$



$$bel(x_t) = \eta P(z_t|x_t)\overline{bel}(x_t)$$



Kalman Filter (KF)

Based on the Bayes Filter (called **Markov localization**):

$$Belief(\mathbf{x}_t) = p(\mathbf{x}_t|\mathbf{u}_1, \mathbf{z}_1 \ \ldots, \mathbf{u}_t, \mathbf{z}_t) = p(\mathbf{x}_t|\mathbf{u}_{1:t}, \mathbf{z}_{1:t})$$

$$= \eta \underbrace{p(\mathbf{z}_t|\mathbf{x}_t)}_{Correction} \underbrace{\int p(\mathbf{x}_t|\mathbf{u}_t, \mathbf{x}_{t-1})\, Belief(\mathbf{x}_{t-1})\ d\mathbf{x}_{t-1}}_{Prediction\ (prior)}$$

KF computes this equation alternating between two steps:

- **Prediction** (computes prior): $\overline{bel}(\mathbf{x}_t) = \int p(\mathbf{x}_t|\mathbf{u}_t, \mathbf{x}_{t-1})\, bel(\mathbf{x}_{t-1})\ d\mathbf{x}_{t-1}$ Total probability (sum rule)
- **Correction** (computes posterior): $bel(\mathbf{x}_t) = \eta p(\mathbf{z}_t|\mathbf{x}_t)\overline{bel}(\mathbf{x}_t)$ Bayes rule

where:

1) The motion and measurement models are **linear**

2) All PDFs involved are **Gaussians**: $p(\mathbf{x}_t|\mathbf{u}_t, \mathbf{x}_{t-1})$, $p(\mathbf{z}_t|\mathbf{x}_t)$, $bel(\mathbf{x}_0)$

KF: Let's go into the details

Kalman Filter is a general tool to estimate the state x of a process that is governed by a linear stochastic **model**:

$$\mathbf{x}_t = A_t\mathbf{x}_{t-1} + B_t\mathbf{u}_t + \boldsymbol{\varepsilon}_t$$

$$\boldsymbol{\varepsilon}_t \sim N(0, R_t)$$

Prediction phase

R_t: covariance of the uncertainty introduced by the motion process

with a linear measurement equation

$$\mathbf{z}_t = C_t\mathbf{x}_t + \boldsymbol{\delta}_t$$

$$\boldsymbol{\delta}_t \sim N(0, Q_t)$$

Correction phase

Q_t: Covariance of the measurement noise

A_t Matrix (nxn) that describes how the state $\mathbf{x}$ evolves from $t-1$ to t without motion command ($\mathbf{u}_t$)

B_t Matrix (nxl) that describes how the motion command $\mathbf{u}_t$ (lx1) changes the state x from $t-1$ to t

C_t Matrix (kxn) that describes how the state $\mathbf{x}_t$ is related to an observation $\mathbf{z}_t$.

$\boldsymbol{\varepsilon}_t$ $\boldsymbol{\delta}_t$ Random vectors representing the process/motion and measurement noise. They are assumed independent, zero-mean Gaussian, with covariance R_t and Q_t, respectively.

Kalman Filter from Bayes filter

Posterior $bel(\mathbf{x}_t) = \eta$ *Likelihood* $p(\mathbf{z}_t|\mathbf{x}_t)$ $\int p(\mathbf{x}_t|\mathbf{u}_t, \mathbf{x}_{t-1})\ bel(\mathbf{x}_{t-1})\ d\mathbf{x}_{t-1}$

$\overline{Bel}(x_t)$ *or prior*

$N(\mathbf{x}_t; \boldsymbol{\mu}_t, \Sigma_t)$

$N(\mathbf{z}_t; C_t\mathbf{x}_t, Q_t)$
$= \mathcal{L}(\mathbf{x}_t)$ ($\mathbf{z}_t$ given)
Gaussian-shaped in $\mathbf{x}_t$

$N(\mathbf{x}_t; \bar{\boldsymbol{\mu}}_t, \bar{\Sigma}_t)$

$= N(\mathbf{x}_t; A_t\mathbf{x}_{t-1} + B_t\mathbf{u}_t, R_t)$

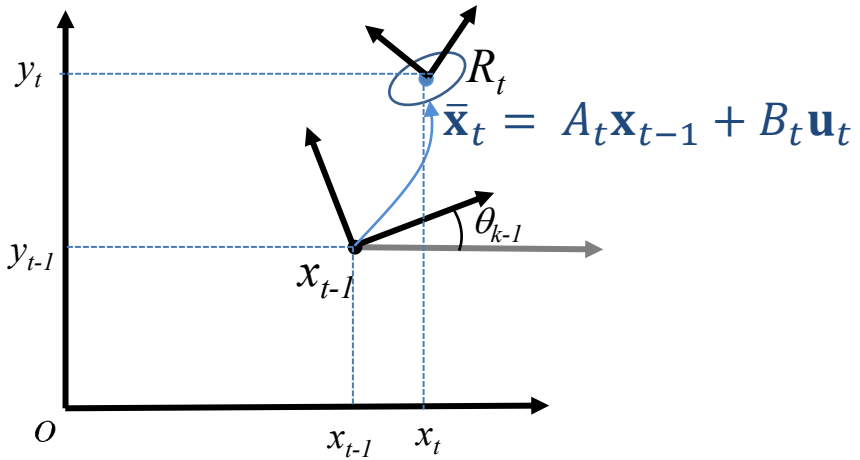
$= N(\mathbf{x}_{t-1}; \boldsymbol{\mu}_{t-1}, \Sigma_{t-1})$

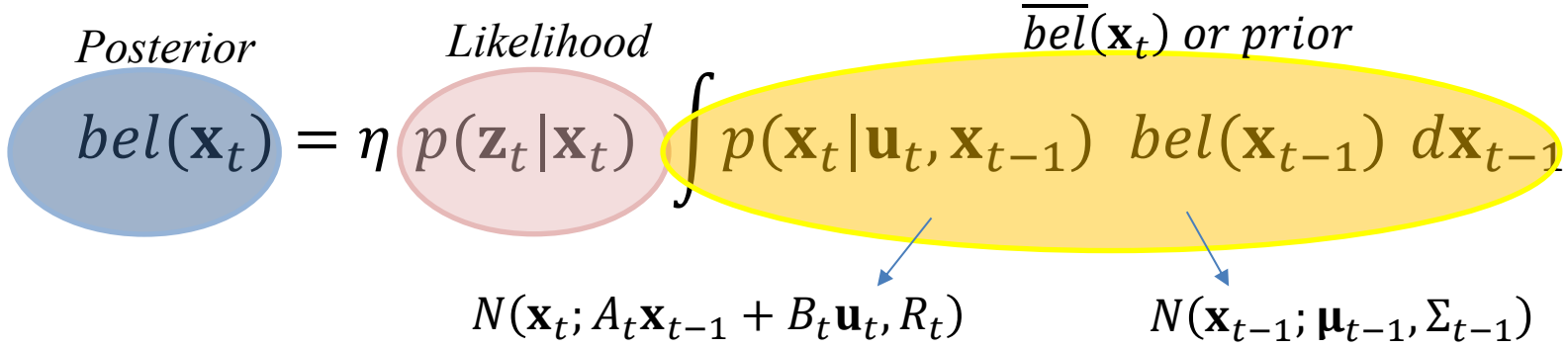
Distribution of previous state $\mathbf{x}_{t-1}$

Product of Gaussians:

Gaussian prior×Gaussian likelihood
$\Longrightarrow$ Gaussian posterior.

Motion model: Distribution of the resulting state $\mathbf{x}_t$, given $\mathbf{u}_t$ and $\mathbf{x}_{t-1}$, due to motion/process uncertainty





Motion model: Distribution of the resulting state $\mathbf{x}_t$, given $\mathbf{u}_t$ and $\mathbf{x}_{t-1}$, due to motion/process uncertainty

Distribution of previous state $\mathbf{x}_{t-1}$

Prediction phase (Predicted belief / prior)

The **predicted belief $\overline{bel}(\mathbf{x}_t)$ is also Gaussian**:

- Product of gaussians: $p(\mathbf{x}_t|\mathbf{u}_t, \mathbf{x}_{t-1})bel(\mathbf{x}_{t-1})$
- Marginalization: $\int p(\mathbf{x}_t, \mathbf{x}_{t-1}|\mathbf{u}_t)\ d\mathbf{x}_{t-1}$

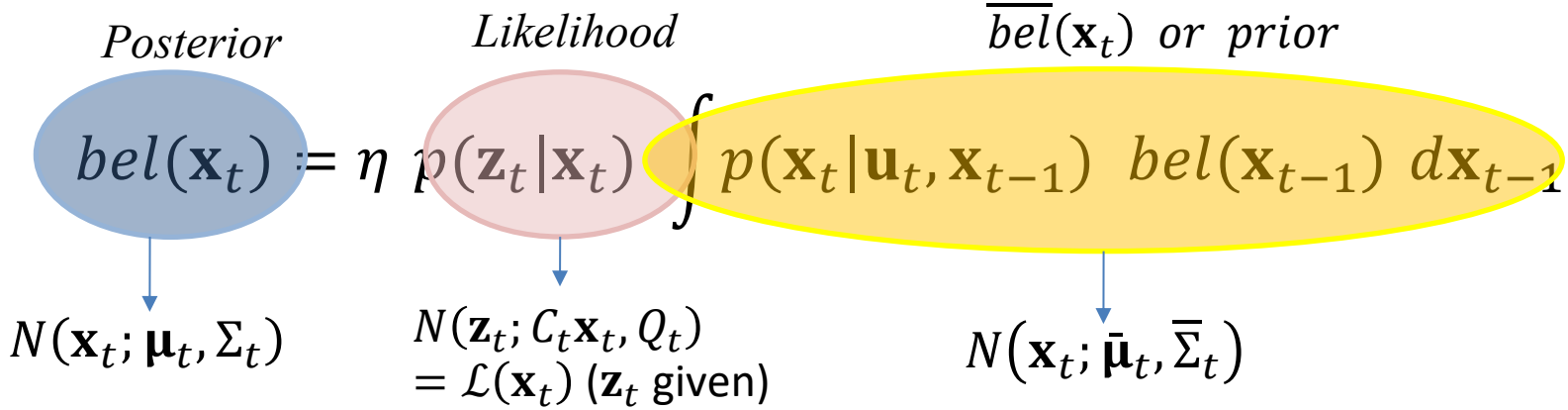
} preserve Gaussianity

$$\overline{bel}(\mathbf{x}_t) = N\left(\mathbf{x}_t; \bar{\boldsymbol{\mu}}_t, \bar{\Sigma}_t\right) \begin{cases} \bar{\boldsymbol{\mu}}_t = A_t\boldsymbol{\mu}_{t-1} + B_t\mathbf{u}_t \\ \bar{\Sigma}_t = A_t\Sigma_{t-1}A_t^T + R_t \end{cases}$$

$\mathbf{u}_t$: known motion command

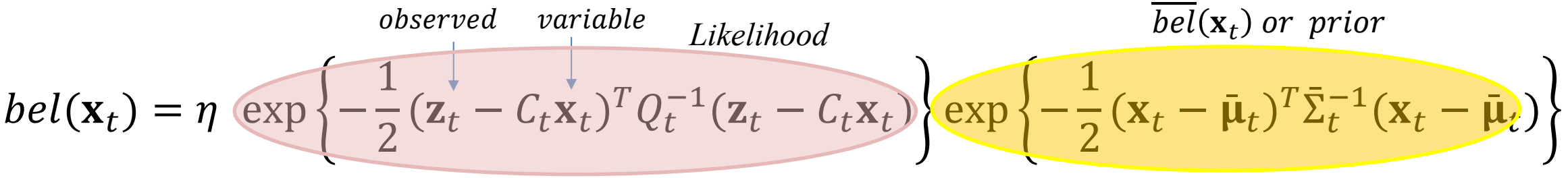
previous state uncertainty propagated through the process

uncertainty introduced by the motion process



Correction phase (posterior computation):

$$z_t = C_t x_t + \delta_t \quad \Rightarrow \quad p(z_t \mid x_t) = N(z_t; C_t x_t, Q_t)$$ Observation model



Gaussian-shaped likelihood × Gaussian prior → Gaussian posterior (after normalization)

prior

$$bel(\mathbf{x}_t) \begin{cases} mean: \ \boldsymbol{\mu}_t = \bar{\boldsymbol{\mu}}_t + K_t(\mathbf{z}_t - C_t\bar{\boldsymbol{\mu}}_t) \\ covariance: \ \Sigma_t = (I - K_t C_t)\bar{\Sigma}_t \end{cases}$$

innovation

with gain $K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1}$

(K_t balances prediction uncertainty against measurement uncertainty

Algorithm **Kalman_filter** $(\boldsymbol{\mu}_{t-1}, \Sigma_{t-1}, \mathbf{u}_t, \mathbf{z}_t)$:

Prediction:

$$\bar{\boldsymbol{\mu}}_t = A_t \boldsymbol{\mu}_{t-1} + B_t \mathbf{u}_t$$

$$\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t$$

Correction:

$$K_t = \bar{\Sigma}_t C_t^T \left(C_t \bar{\Sigma}_t C_t^T + Q_t\right)^{-1}$$

$$\boldsymbol{\mu}_t = \bar{\boldsymbol{\mu}}_t + K_t \left(\mathbf{z}_t - C_t \bar{\boldsymbol{\mu}}_t\right)$$

$$\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t$$

Return $\boldsymbol{\mu}_t, \Sigma_t$

The uncertainty increases: the previous state uncertainty is propagated through the process, and new motion/process uncertainty R_t is added.

Covariance of the **Innovation**

Innovation: discrepancy between the observation $\mathbf{z}_t$ and the predicted measurement $C_t \bar{\boldsymbol{\mu}}_t$

Correction does not increase the predicted covariance: $\det(\Sigma_t) \leq \det(\bar{\Sigma}_t)$

Kalman correction = MAP estimation

Two views of the same linear–Gaussian update

Predicted state $x_t \sim \mathcal{N}(\bar{\mu}_t, \bar{\Sigma}_t)$

Measurement model $z_t = C_t x_t + v_t,\ v_t \sim \mathcal{N}(0, Q_t)$

MAP = weighted least squares with a prior

$$J(x) = \underbrace{\frac{1}{2}(x - \bar{\mu}_t)^\top \bar{\Sigma}_t^{-1}(x - \bar{\mu}_t)}_{\text{Trust the prediction}} + \underbrace{\frac{1}{2}(z_t - C_t x)^\top Q_t^{-1}(z_t - C_t x)}_{\text{Fit the measurement}}$$

$$\hat{x}_{\text{MAP}} = \arg\min_x J(x)$$

Same estimate

$$\mu_t = \hat{x}_{\text{MAP}}$$

Gaussian posterior: mean = mode

Same uncertainty

$$\Sigma_t = \left[(\bar{\Sigma}_t)^{-1} + C_t^\top Q_t^{-1} C_t\right]^{-1}$$

Posterior covariance after combining prediction and measurement

Kalman filtering computes the update recursively. MAP expresses it as an optimization.

Exact for linear–Gaussian models with positive-definite covariances; generally approximate for nonlinear models.

Kalman Filter — Summary

Summary

- Computationally efficient: only the mean vector and covariance matrix need to be propagated.
- Exact Bayes filter solution for linear-Gaussian systems.
- The posterior mean is the optimal minimum-MSE estimate.
- Most robotics motion and sensing models are nonlinear, so the standard KF cannot be directly applied.

BUT we can **locally linearize** the nonlinear motion and sensing models → **Extended Kalman Filter**

Extended Kalman Filter

EKF extends the KF to nonlinear motion and measurement models.

Prediction): $\mathbf{x}_t = \mathbf{g}(\mathbf{x}_{t-1}, \mathbf{u}_t) + \boldsymbol{\varepsilon}_t$ (Instead of $\mathbf{x}_t = A_t\mathbf{x}_{t-1} + B_t\mathbf{u}_t + \boldsymbol{\varepsilon}_t$)

Correction: $\mathbf{z}_t = \mathbf{h}(\mathbf{x}_t) + \boldsymbol{\delta}_t$ (Instead of $\mathbf{z}_t = C_t\mathbf{x}_t + \boldsymbol{\delta}_t$)

PROBLEM

Nonlinear transformations of Gaussian RVs are generally not Gaussian



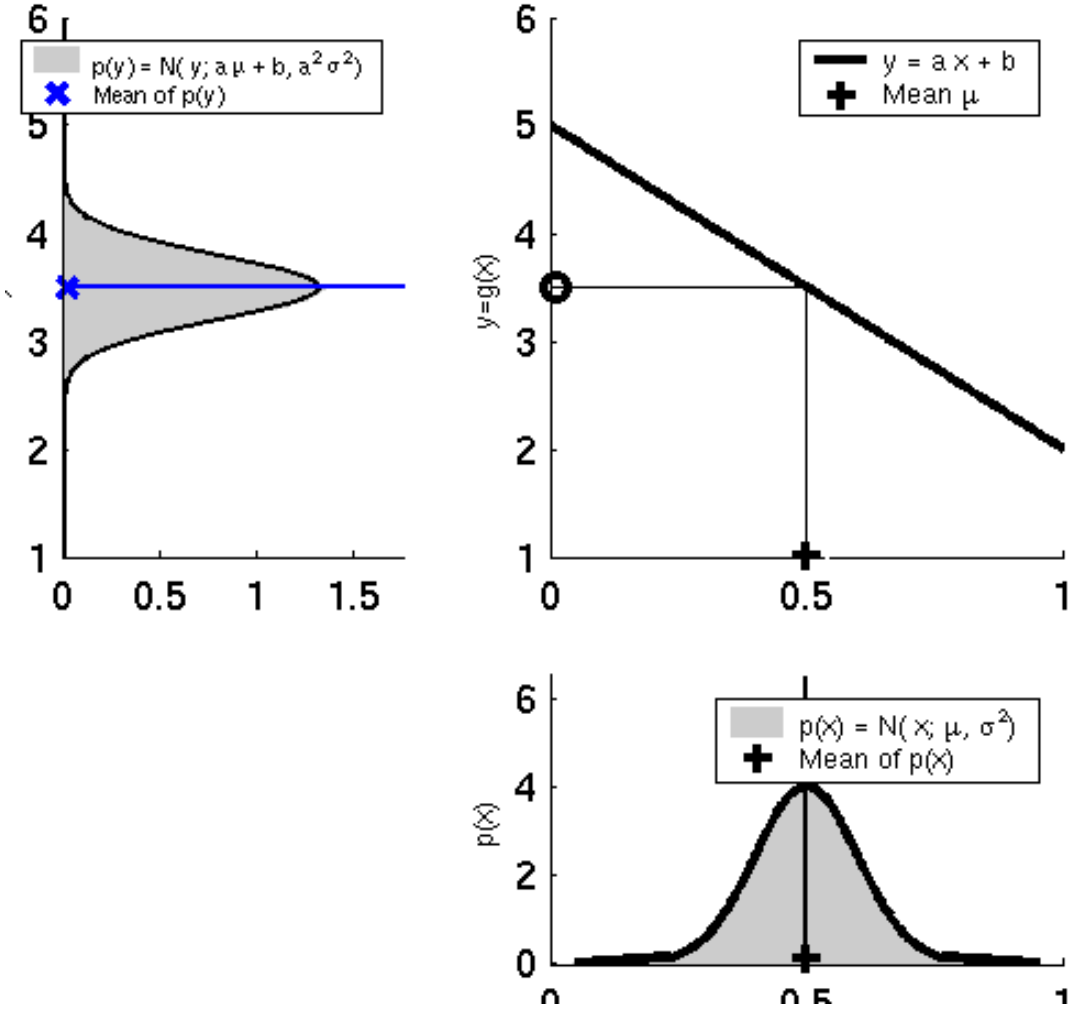
Mean and covariance are no longer sufficient to exactly represent the transformed distribution

EKF solution:

Locally linearize the nonlinear models and approximate the belief as Gaussian.

Mean and Covariance propagation

Linear transformations preserve Gaussianity and propagate covariance exactly



For scalar RVs (as in the plot):

$$\left.\begin{aligned} X &\sim N(\mu, \sigma^2) \\ Y &= aX + b \end{aligned}\right\} \quad \Rightarrow \quad Y \sim N(a\mu + b, a^2\sigma^2)$$

For vector RVs:

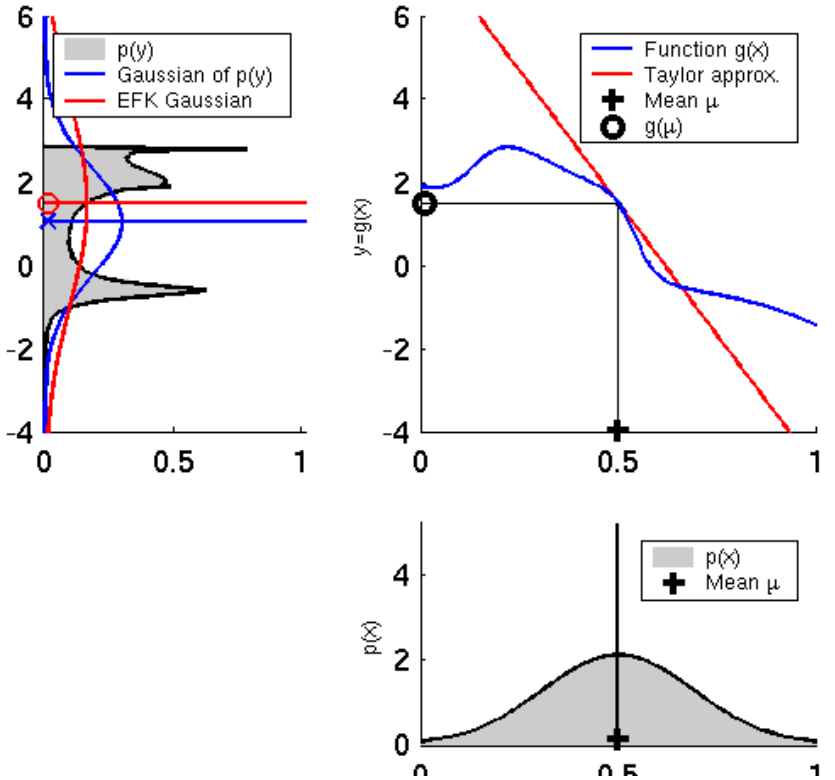
$$\left.\begin{aligned} X &\sim N(\mathbf{\mu}, \Sigma) \\ Y &= AX + \mathbf{b} \end{aligned}\right\} \Rightarrow Y \sim N(\underbrace{A\mathbf{\mu} + \mathbf{b}}_{E[Y]}, \underbrace{A\Sigma A^T}_{\mathrm{Cov}(Y)})$$

mean propagation and **covariance propagation**

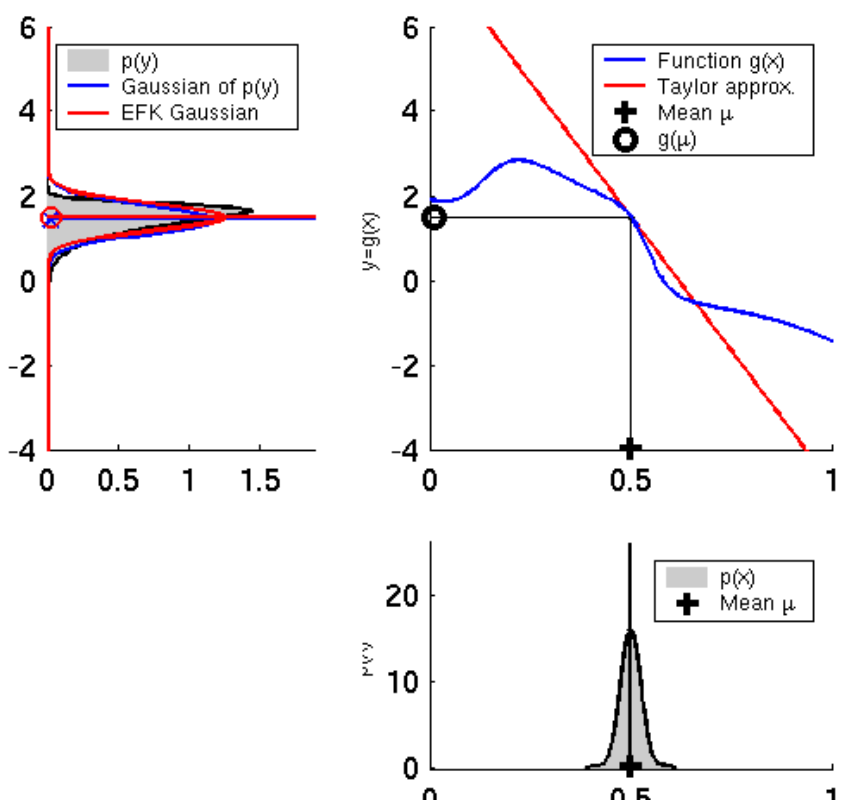
From: Thrun *et.al.*

Mean and Covariance propagation

A nonlinear transformation of a Gaussian is generally non-Gaussian.



Large variance of x



Small variance of x

EKF works best when the **uncertainty is small** enough that the nonlinear function is approximately linear over the region of significant probability

From: Thrun *et.al.*

EKF Linearization: First Order Taylor Expansion

- **Prediction:** $\mathbf{x}_t = \mathbf{g}(\mathbf{x}_{t-1}, \mathbf{u}_t) + \boldsymbol{\varepsilon}_t = \mathbf{x}_{t-1} \oplus \mathbf{u}_t + \boldsymbol{\varepsilon}_t$

$$\mathbf{g}(\mathbf{x}_{t-1}, \mathbf{u}_t) \approx \mathbf{g}(\boldsymbol{\mu}_{t-1}, \mathbf{u}_t) + \left.\frac{\partial g(\mathbf{x}_{t-1}, \mathbf{u}_t)}{\partial \mathbf{x}_{t-1}}\right|_{\mathbf{x}_{t-1}=\boldsymbol{\mu}_t-1} (\mathbf{x}_{t-1} - \boldsymbol{\mu}_{t-1}) = \mathbf{g}(\boldsymbol{\mu}_{t-1}, \mathbf{u}_t) + G_t\ \Delta\mathbf{x}_{t-1}$$

Linearization for $\mathbf{x}_{t-1}$ at the mean $\boldsymbol{\mu}_{t-1}$

Jacobian G_t evaluated at $\mathbf{x}_{t-1} = \boldsymbol{\mu}_{t-1}$

$\Delta\mathbf{x}_{t-1}$

$$\mathbf{x}_t = \mathbf{g}(\mathbf{x}_{t-1}, \mathbf{u}_t) + \boldsymbol{\varepsilon}_t \approx \underbrace{\mathbf{g}(\boldsymbol{\mu}_{t-1}, \mathbf{u}_t)}_{\boldsymbol{\mu}_{t-1} \oplus \mathbf{u}_t = \bar{\boldsymbol{\mu}}_t} + G_t\ \Delta\mathbf{x}_{t-1} + \boldsymbol{\varepsilon}_t$$

Random variables: $\Delta\mathbf{x}_{t-1} \sim N(0, \Sigma_{t-1})$, $\boldsymbol{\varepsilon}_t \sim N(0, R_t)$

Parameters of the Gaussian approximation of the predicted belief:

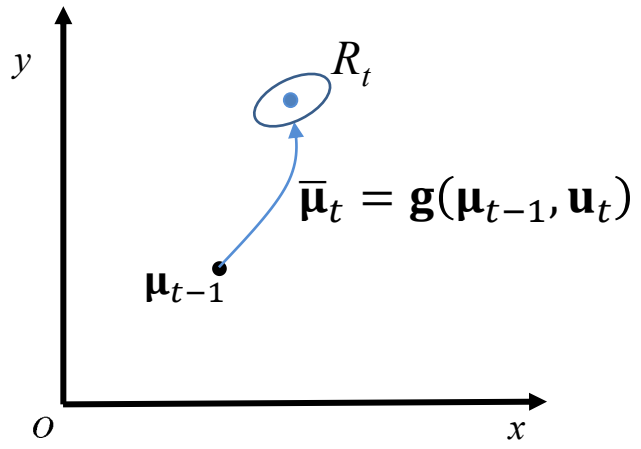
$$\overline{bel}(\mathbf{x}_t) \approx \mathcal{N}(\mathbf{x}_t; \bar{\boldsymbol{\mu}}_t, \bar{\Sigma}_t)$$

$$\bar{\boldsymbol{\mu}}_t = \mathbf{g}(\boldsymbol{\mu}_{t-1}, \mathbf{u}_t) \quad \text{Predicted mean}$$

$$\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t \quad \text{Predicted Covariance}$$

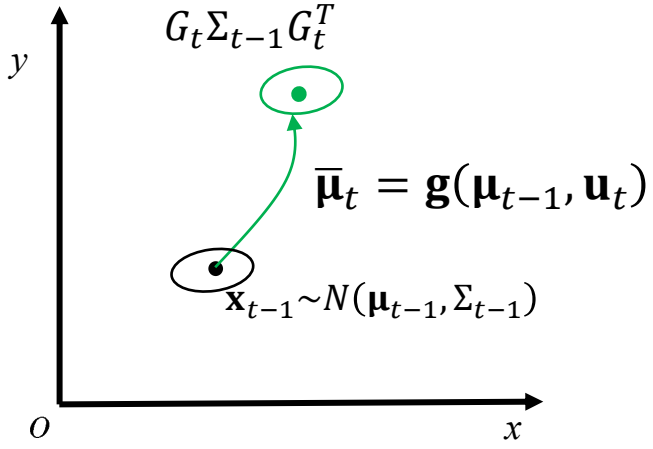
EKF Prediction Step

Uncertainty introduced by motion/process noise



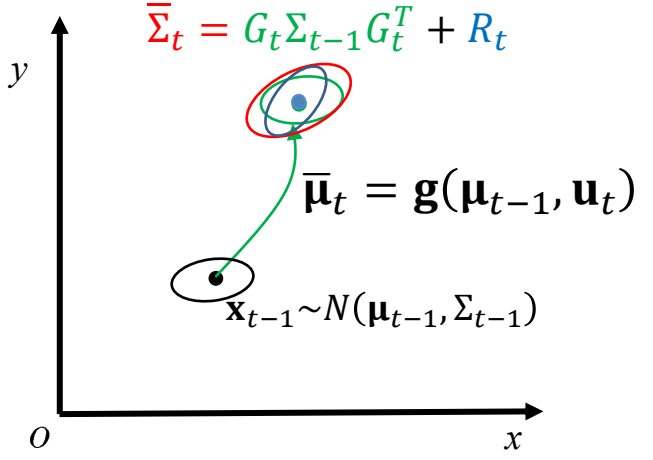
Motion of a **known pose $\boldsymbol{\mu}_{t-1}$** with process-noise covariance R_t

Propagation of previous pose uncertainty



Motion of the pose $\mathbf{x}_{t-1} \sim N(\boldsymbol{\mu}_{t-1}, \Sigma_{t-1})$ by a **known motion $\mathbf{u}_t$**

Combination of both uncertainty sources



Prediction with **both** previous-state uncertainty and process uncertainty

EKF Linearization: First Order Taylor Expansion

- **Correction:**

$$\mathbf{h}(\mathbf{x}_t) \approx \mathbf{h}(\bar{\boldsymbol{\mu}}_t) + \left.\frac{\partial \boldsymbol{h}(\mathbf{x})}{\partial \mathbf{x}}\right|_{\mathbf{x}=\bar{\boldsymbol{\mu}}_t} (\mathbf{x}_t - \bar{\boldsymbol{\mu}}_t) \quad = \mathbf{h}(\bar{\boldsymbol{\mu}}_t) + H_t(\mathbf{x}_t - \bar{\boldsymbol{\mu}}_t)$$

$$H_t = \left.\frac{\partial \mathbf{h}(\mathbf{x})}{\partial \mathbf{x}}\right|_{\mathbf{x}=\bar{\boldsymbol{\mu}}_t}$$

Predicted mean computed in the prediction step $\bar{\boldsymbol{\mu}}_t = \mathbf{g}(\boldsymbol{\mu}_{t-1}, \mathbf{u}_t)$

Therefore, the **approximate likelihood** is:

$$p(\mathbf{z}_t \mid \mathbf{x}_t) \approx \mathcal{N}(\mathbf{z}_t; \mathbf{h}(\bar{\boldsymbol{\mu}}_t) + H_t(\mathbf{x}_t - \bar{\boldsymbol{\mu}}_t), Q_t)$$ Gaussian-shaped in $\mathbf{x}_t$

Predicted belief: $\overline{bel}(\mathbf{x}_t) \approx \mathcal{N}(\mathbf{x}_t; \bar{\boldsymbol{\mu}}_t, \bar{\Sigma}_t)$

Multiplication of two Gaussians

$$bel(\mathbf{x}_t) \propto p(\mathbf{z}_t \mid \mathbf{x}_t)\, \overline{bel}(\mathbf{x}_t) \approx \mathcal{N}(\mathbf{x}_t; \boldsymbol{\mu}_t, \Sigma_t)$$

Innovation

$$\boldsymbol{\mu}_t = \bar{\boldsymbol{\mu}}_t + K_t(\overbrace{\mathbf{z}_t - \mathbf{h}(\bar{\boldsymbol{\mu}}_t)})$$

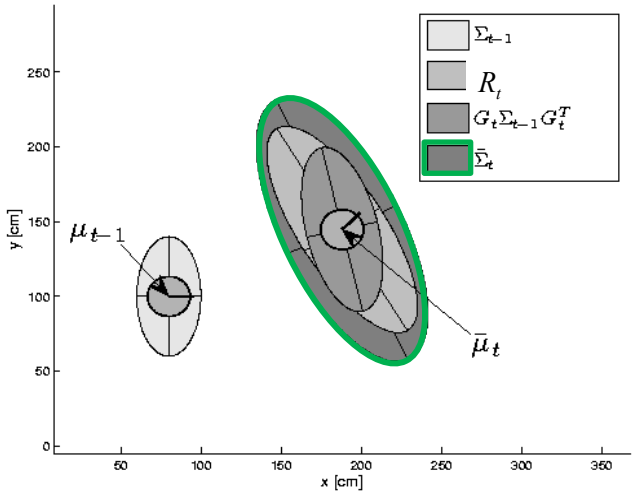
$$\Sigma_t = (I - K_t H_t)\bar{\Sigma}_t$$

Correction does not increase covariance

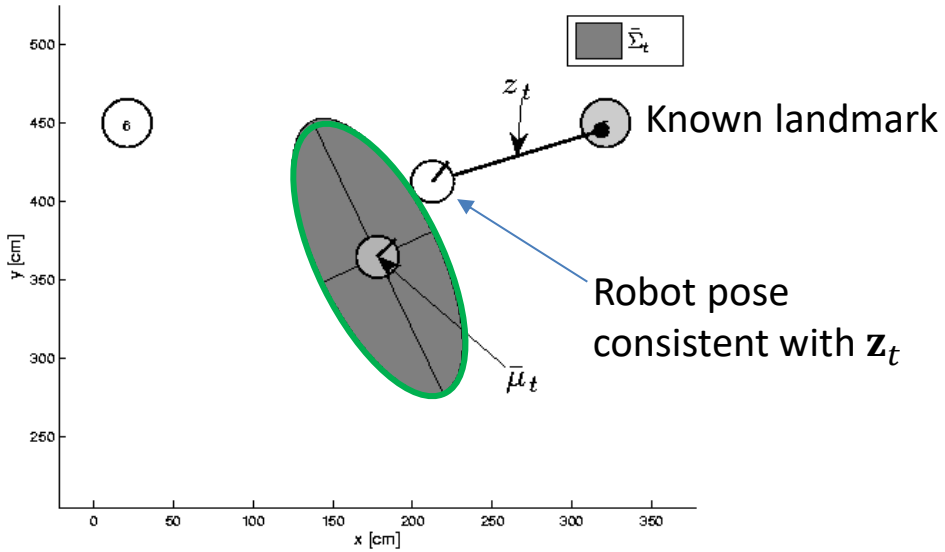
$$K_t = \bar{\Sigma}_t H_t^T (H_t \bar{\Sigma}_t H_t^T + Q_t)^{-1}$$

EKF Correction Step

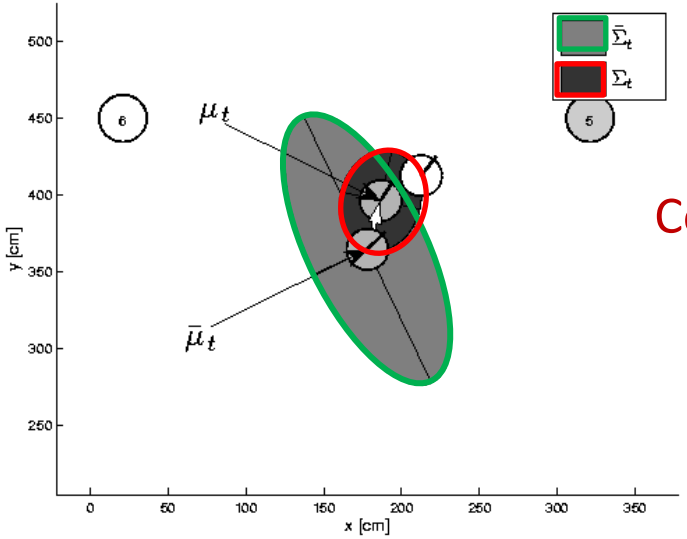
Predicted belief



Take observation z_t



Corrected belief



Correction

$$\underbrace{\overline{bel}(\mathbf{x}_t)}_{\text{prediction}} \times \underbrace{p(\mathbf{z}_t \mid \mathbf{x}_t)}_{\text{likelihood}} \longrightarrow \underbrace{bel(\mathbf{x}_t)}_{\text{posterior}} .$$

From: Thrun *et.al.*

Extended_Kalman_filter $(\boldsymbol{\mu}_{t-1}, \Sigma_{t-1}, \mathbf{u}_t, \mathbf{z}_t)$:

Prediction:

1. $\bar{\boldsymbol{\mu}}_t = \mathbf{g}(\boldsymbol{\mu}_{t-1}, \mathbf{u}_t) = \boldsymbol{\mu}_{t-1} \oplus \mathbf{u}_t$
2. $\overline{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t$

Correction:

1. $K_t = \overline{\Sigma}_t H_t^T (H_t \overline{\Sigma}_t H_t^T + Q_t)^{-1}$
2. $\boldsymbol{\mu}_t = \bar{\boldsymbol{\mu}}_t + K_t(\underbrace{\mathbf{z}_t - \mathbf{h}(\bar{\boldsymbol{\mu}}_t)}_{innovation})$
3. $\Sigma_t = (I - K_t H_t)\overline{\Sigma}_t$

Return $\boldsymbol{\mu}_t, \Sigma_t$

Jacobians

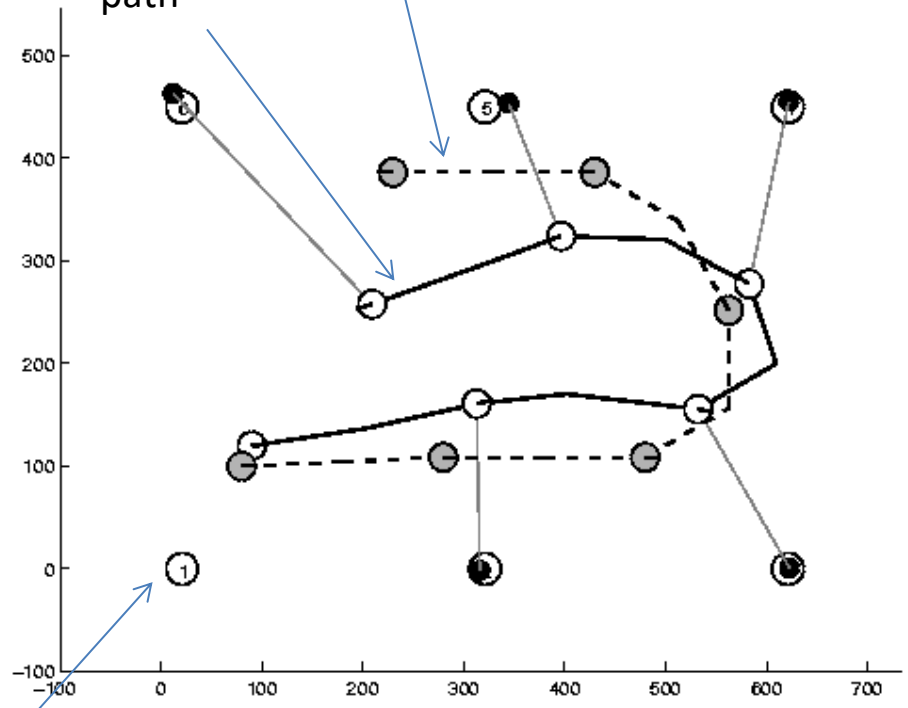
$$G_t = \left.\frac{\partial \mathbf{g}(\mathbf{x}, \mathbf{u}_t)}{\partial \mathbf{x}}\right|_{\mathbf{x}=\boldsymbol{\mu}_{t-1}}$$

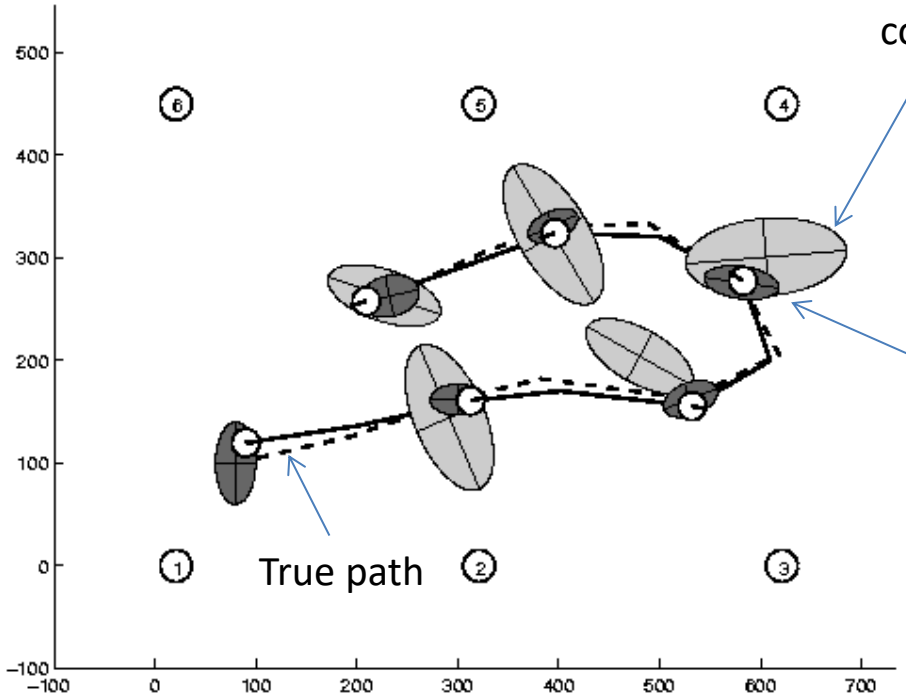
$$H_t = \left.\frac{\partial \mathbf{h}(\mathbf{x})}{\partial \mathbf{x}}\right|_{\mathbf{x}=\bar{\boldsymbol{\mu}}_t}.$$

Same structure as the Kalman Filter, using the nonlinear models $\mathbf{g}, \mathbf{h}$ and their local Jacobians G_t, H_t

Extended Kalman Filter

Estimation Sequence





From: Thrun *et.al.*

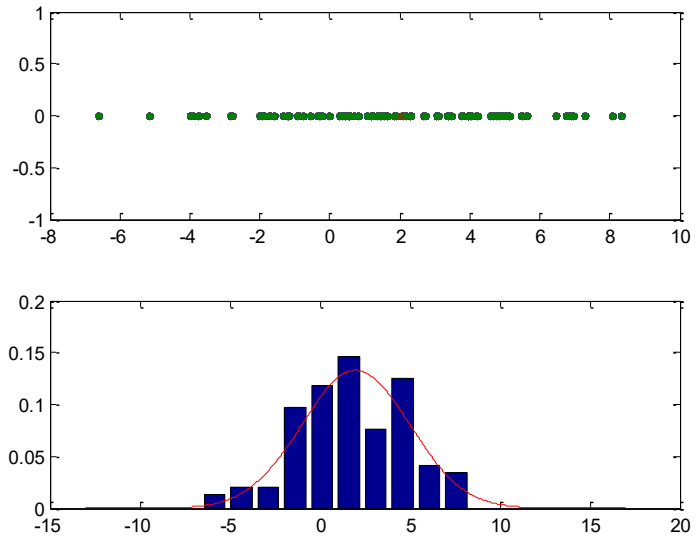
We'll program this example in the practical sessions

Particle Filter

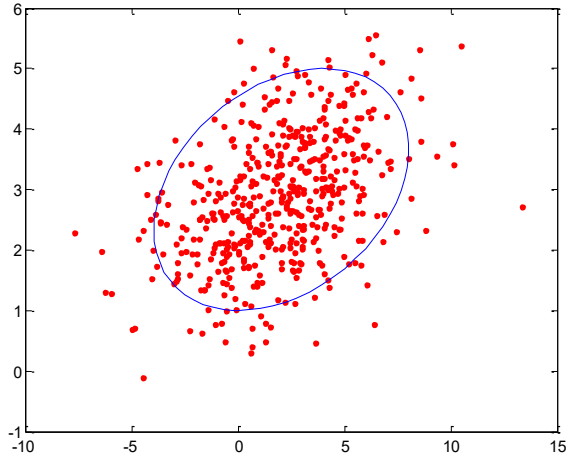
- Monte Carlo implementation of the recursive Bayes filter
- The posterior distribution is approximated by a set of random samples (particles)

Example: Particles for a Gaussian pdf:

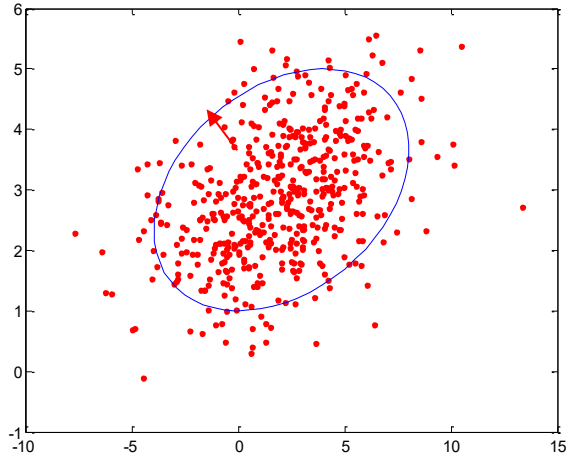
1D



2D



2D pose particles also have an orientation



Unlike KF/EKF, this representation does not have to be Gaussian

Particle Filter

Still an implementation of the Bayes Filter with 2+1 steps

Prediction: $\overline{bel}(\mathbf{x}_t) = \int p\left(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{u}_t\right) bel(\mathbf{x}_{t-1})\, d\mathbf{x}_{t-1}$

$$\mathbf{x}_t^{[i]} \sim p\left(\mathbf{x}_t \mid \mathbf{x}_{t-1}^{[i]}, \mathbf{u}_t\right), \quad i = 1, \ldots, M$$

Approximate that integral by drawing particles from the transition distribution

Correction: $bel(\mathbf{x}_t) = \eta p(\mathbf{z}_t|\mathbf{x}_t)\overline{bel}(\mathbf{x}_t)$

$$w_t^{[i]} \propto p\left(\mathbf{z}_t \mid \mathbf{x}_t^{[i]}\right)$$

Evaluate $p\left(\mathbf{z}_t|\mathbf{x}_t^{[i]}\right)$ for every particle

Resampling: $bel(\mathbf{x}_t)$

$$\tilde{w}_t^{[i]} = \frac{1}{M}$$

Draw *M* particles with replacement according to $w_t^{[i]}$

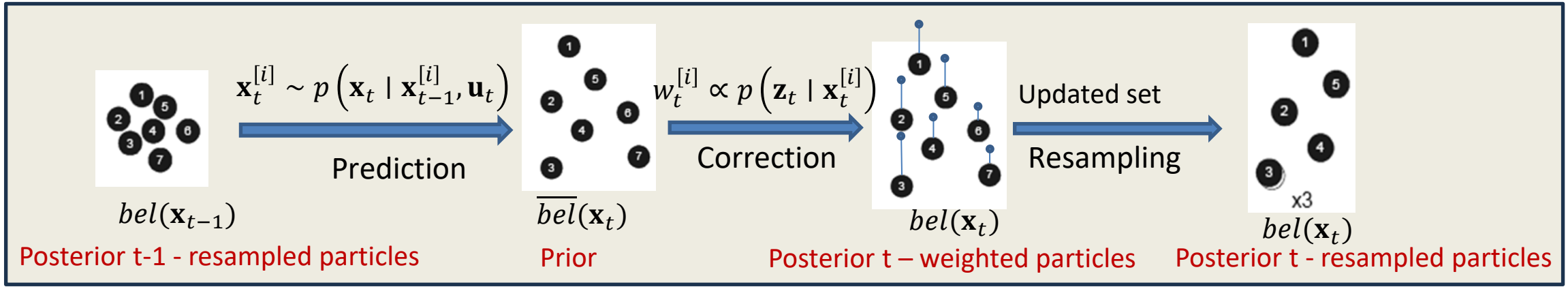
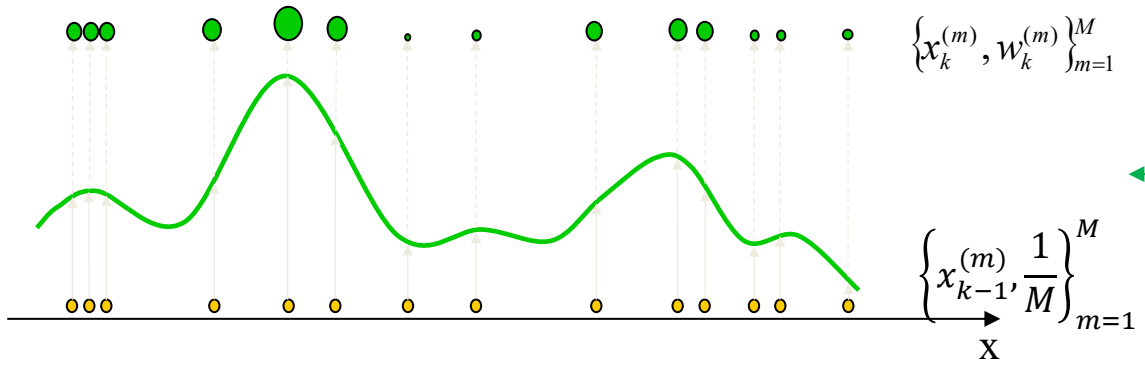


Illustration of the PF in 1D: SIR implementation, M particles: $\left\{x_{k-1}^{(m)}, w^{(m)}\right\}_{m=1}^{M}$

Weights here are represented by the area of the circle

Pose of the particle m at time k

Weight of the particle m



Posterior k (before resampling)

Observation k

Prior time k (all the M particles have the same weight $\frac{1}{M}$

SIR: Sequential Importance Resampling

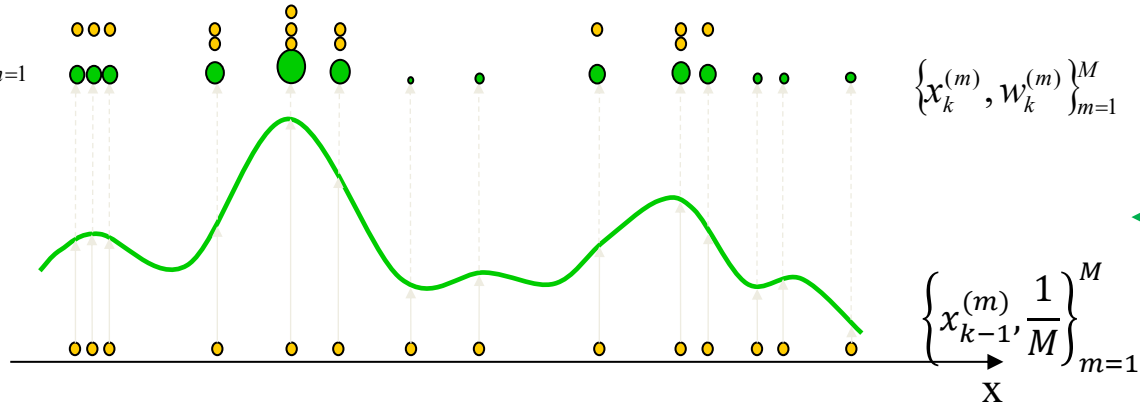
Illustration of the PF in 1D: SIR implementation, M particles: $\left\{x_{k-1}^{(m)}, w^{(m)}\right\}_{m=1}^{M}$

Weights here are represented by the area of the circle

Pose of the particle m at time k

Weight of the particle m

Posterior k (after resampling) $\left\{\tilde{x}_k^{(m)}, \frac{1}{M}\right\}_{m=1}^{M}$



Posterior k (before resampling)

Observation k

Prior time k (all the M particles have the same weight $\frac{1}{M}$

SIR: Sequential Importance Resampling

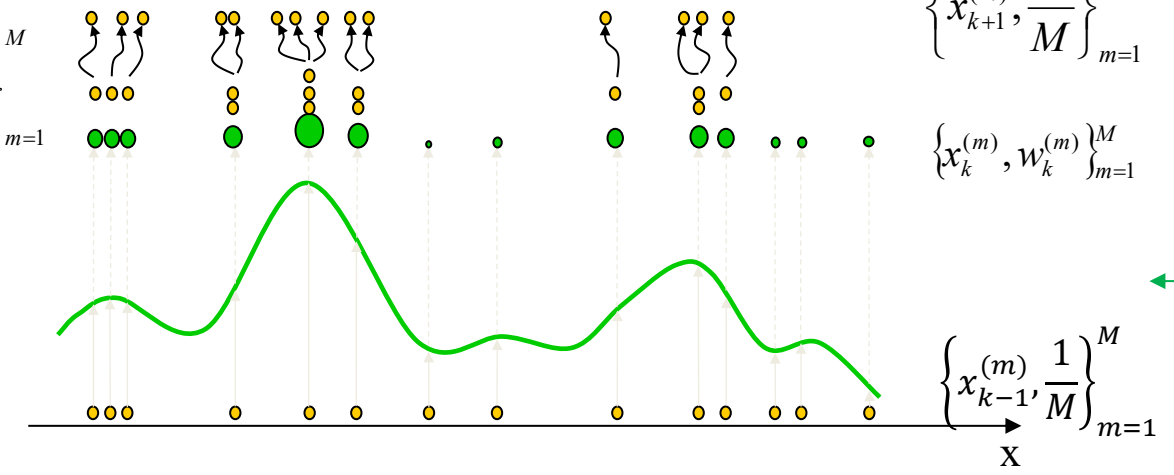
Illustration of the PF in 1D: SIR implementation, M particles: $\left\{x_{k-1}^{(m)}, w^{(m)}\right\}_{m=1}^{M}$

Weights here are represented by the area of the circle

Pose of the particle m at time k

Weight of the particle m

Posterior k
(after resampling) $\left\{\tilde{x}_k^{(m)}, \frac{1}{M}\right\}_{m=1}^{M}$



Prior $k+1$: moved with $g(u_{k+1}, x_k)$

Posterior k (before resampling)

Observation k

Prior time k (all the M particles have the same weight $\frac{1}{M}$

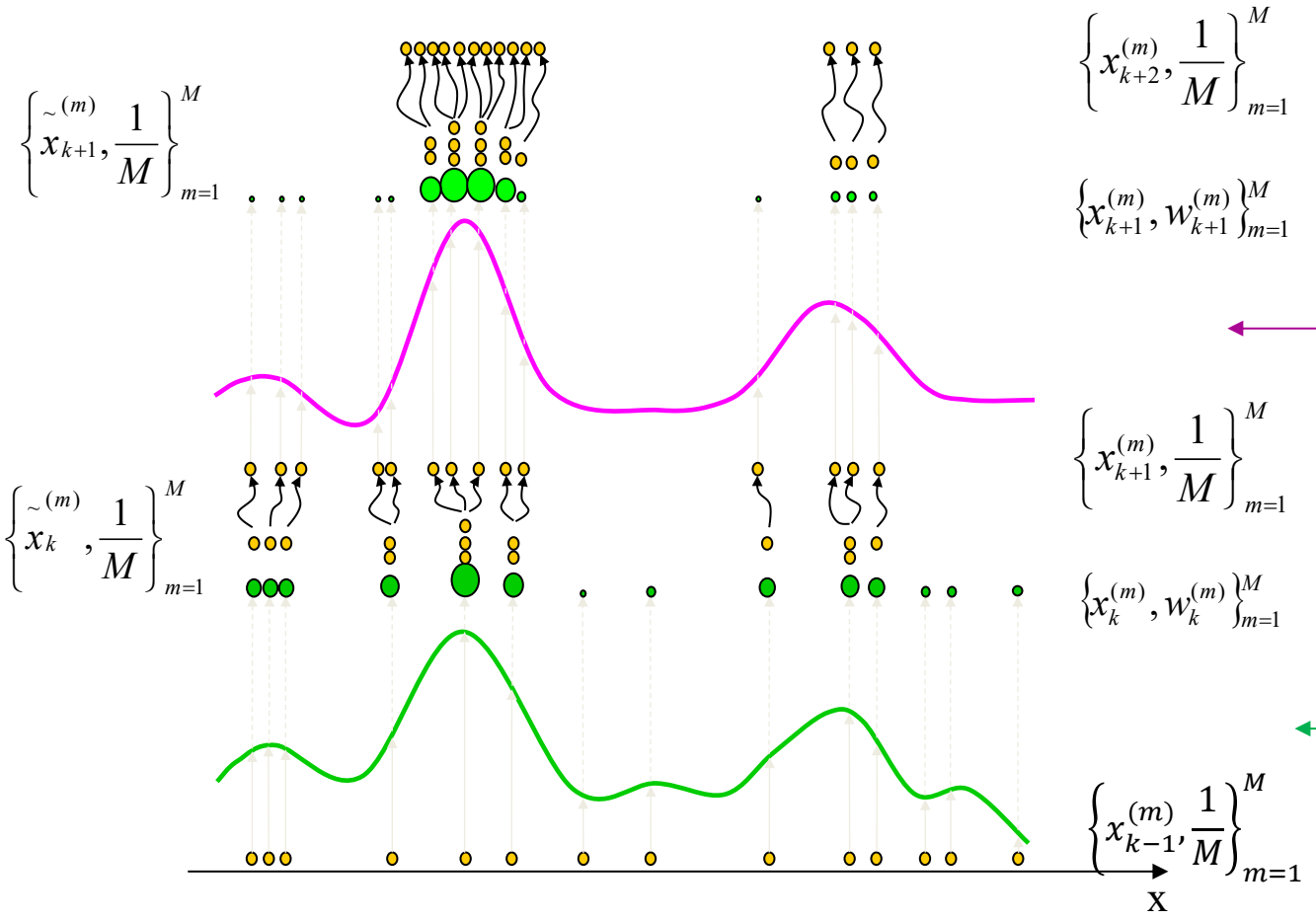
SIR: Sequential Importance Resampling

Illustration of the PF in 1D: SIR implementation, M particles: $\left\{x_{k-1}^{(m)}, w^{(m)}\right\}_{m=1}^{M}$

Pose of the particle m at time k

Weight of the particle m

Weights here are represented by the area of the circle



Posterior k+1 (after resampling) $\left\{\tilde{x}_{k+1}^{(m)}, \frac{1}{M}\right\}_{m=1}^{M}$

$\left\{x_{k+2}^{(m)}, \frac{1}{M}\right\}_{m=1}^{M}$ Prior k+2: moved with $g(u_{k+2}, x_{k+1})$

$\left\{x_{k+1}^{(m)}, w_{k+1}^{(m)}\right\}_{m=1}^{M}$ Posterior k+1 (before resampling)

Observation k+1

Posterior k (after resampling) $\left\{\tilde{x}_{k}^{(m)}, \frac{1}{M}\right\}_{m=1}^{M}$

$\left\{x_{k+1}^{(m)}, \frac{1}{M}\right\}_{m=1}^{M}$ Prior $k+1$: moved with $g(u_{k+1}, x_k)$

$\left\{x_{k}^{(m)}, w_{k}^{(m)}\right\}_{m=1}^{M}$ Posterior k (before resampling)

Observation k

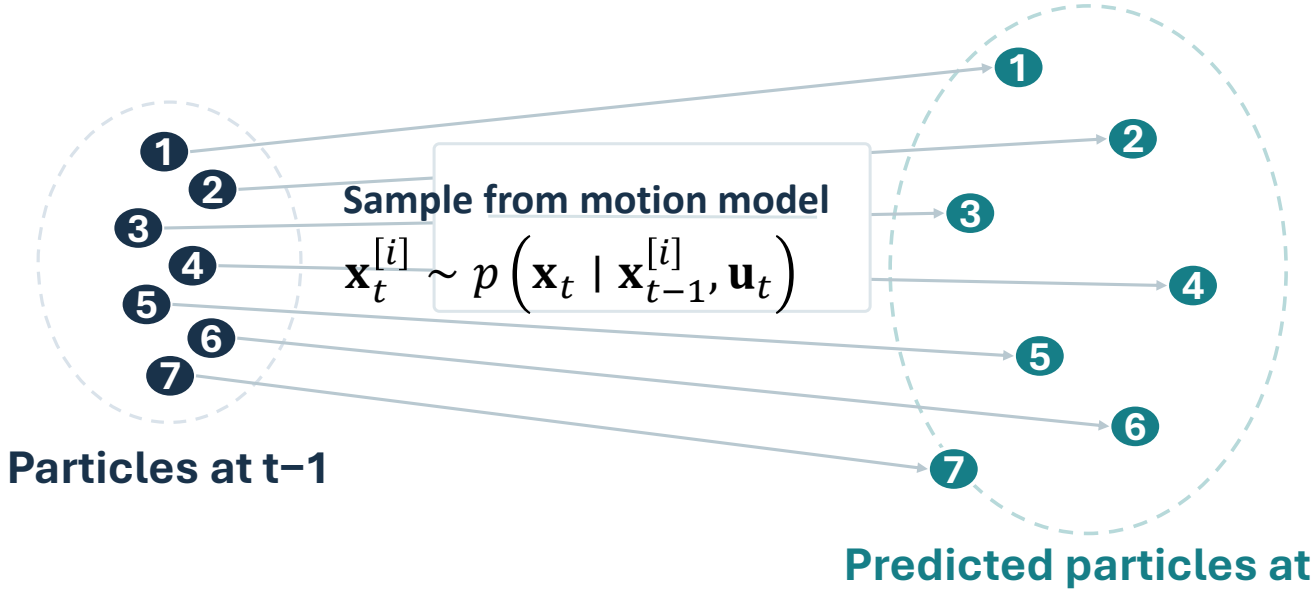
$\left\{x_{k-1}^{(m)}, \frac{1}{M}\right\}_{m=1}^{M}$ Prior time k (all the M particles have the same weight $\frac{1}{M}$

SIR: Sequential Importance Resampling

Particle Filter

1. Prediction

Implements the prediction $\overline{bel}(\mathbf{x}_t)$ by sampling from the prediction model



Motion model definition

```python
def sample_motion_model(x, u, R):
    # x: (3,) previous particle pose [x, y, theta]
    # u: (3,) nominal motion increment [dx, dy, dtheta]
    # R: (3,3) covariance of the motion/process uncertainty
    # returns: (3,) sampled new particle pose

    noise = np.random.multivariate_normal(np.zeros(3), R)
    return compose(x, u + noise)
```

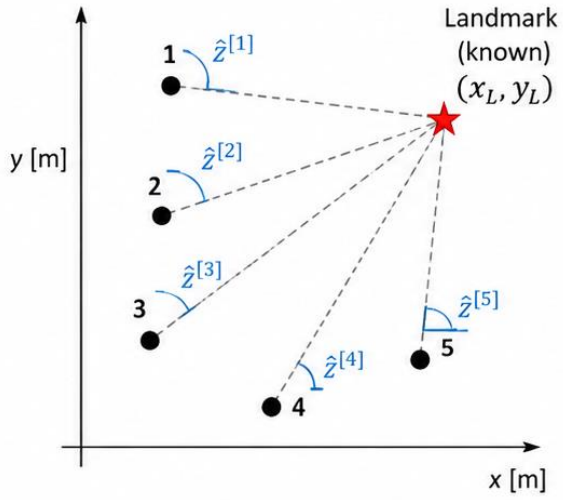
Sample from motion model

```python
# particles: (N,3) array containing the N particle poses
# u_t: (3,) nominal motion increment at time t
# R_t: (3,3) covariance of the motion/process uncertainty

for i in range(N):
    particles[i] = sample_motion_model(particles[i], u_t, R_t)
```

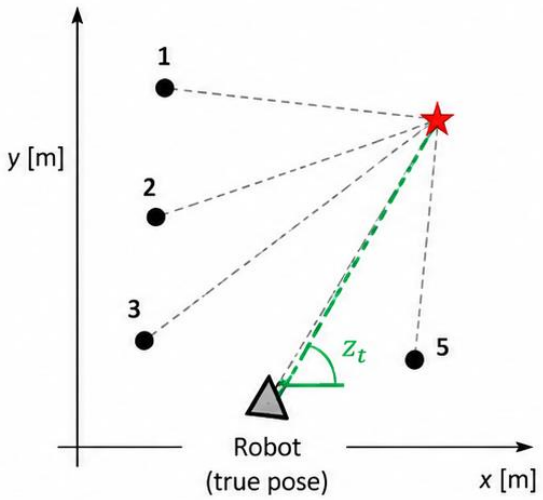
2. Correction: compute particle weights (example for a landmark bearing sensor)

(1) Predicted observation for each particle



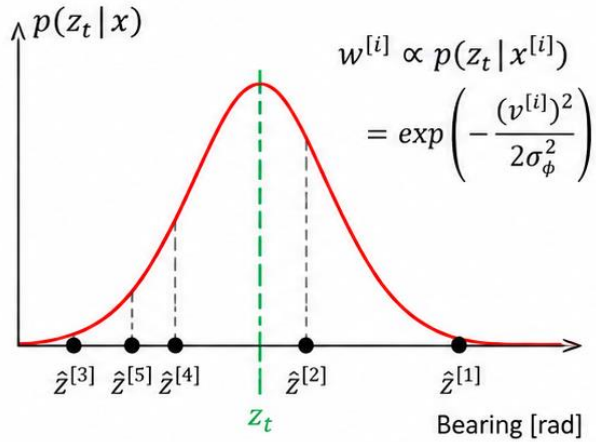
Predicted bearing $\hat{z}^{[i]} = h(x^{[i]})$ to the landmark

(2) Compare with the actual observation



Innovation for each particle: $v^{[i]} = z_t - \hat{z}^{[i]}$

(3) Compute particle likelihood (weight)



Particles with predicted bearing closer to z_t get higher weight

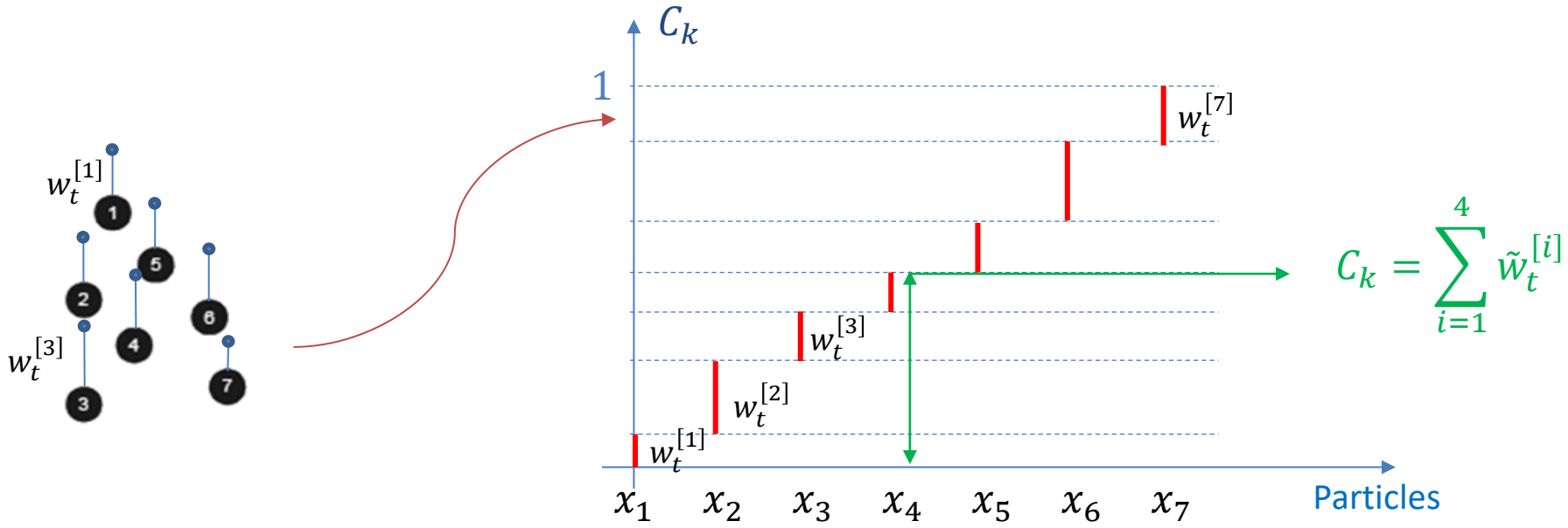
```
# particles: (N,3) predicted poses [x,y,theta]
# landmark: (2,) known position [xL,yL]
# z_t: observed range; sigma_z: sensor noise std.
# weights: (N,) particle weights
for i in range(N):
    z_pred = range_sensor_model(particles[i], landmark) # predicted observation
    innov = z_t - z_pred                                # innovation
    weights[i] = np.exp(-0.5 * innov**2 / sigma_z**2) # likelihood
weights /= np.sum(weights)                              # normalize
```

3. Resampling based on weights (survival of the fittest)

IDEA: Particles with larger weights occupy larger intervals of the cumulative distribution.

b1) Build the cumulative distribution of the normalized particle weights

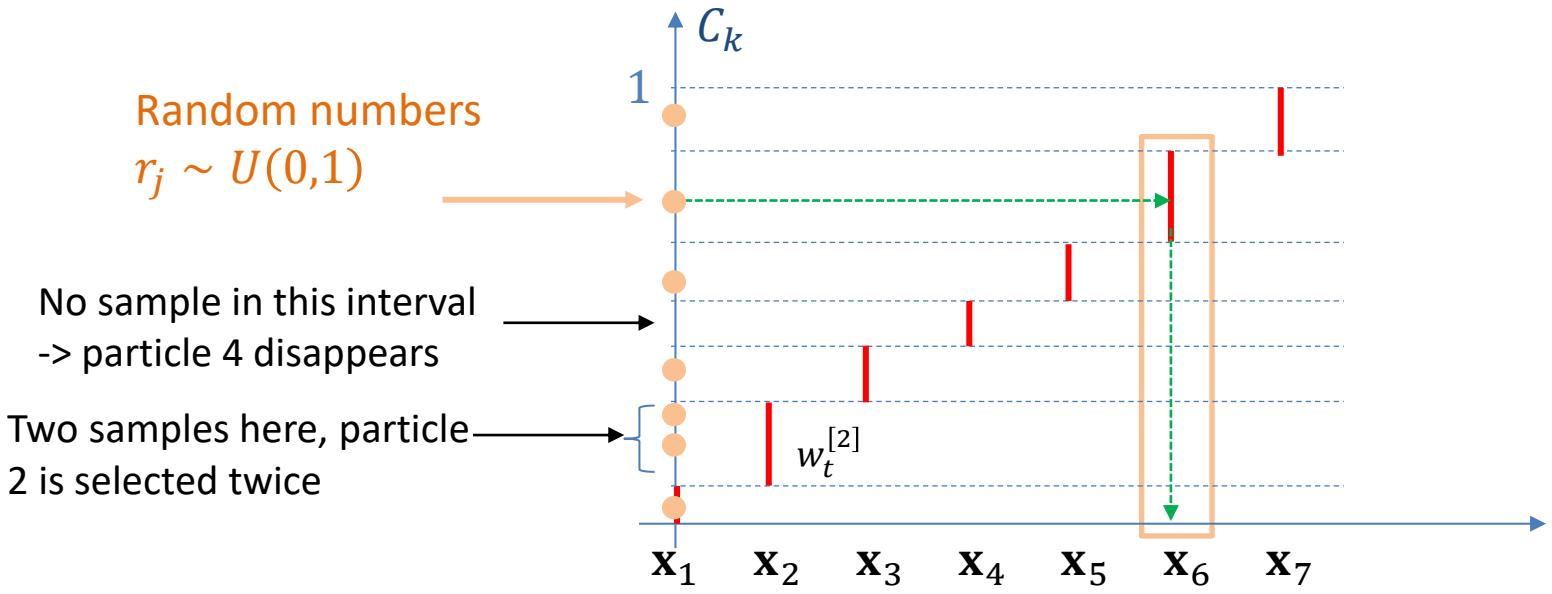
Cumulative distribution: $C_k = \sum_{i=1}^{k} \tilde{w}_t^{[i]}, \quad C_N = 1.$ Sum of the cumulative weights of the particles



3. Resampling based on weights (survival of the fittest)

b2) Randomly generate new particles from the weights

1. Draw N random numbers: $r_j \sim U(0,1),\ j = 1, \ldots, N$
2. For each r_j select the particle whose cumulative-weight interval contains it

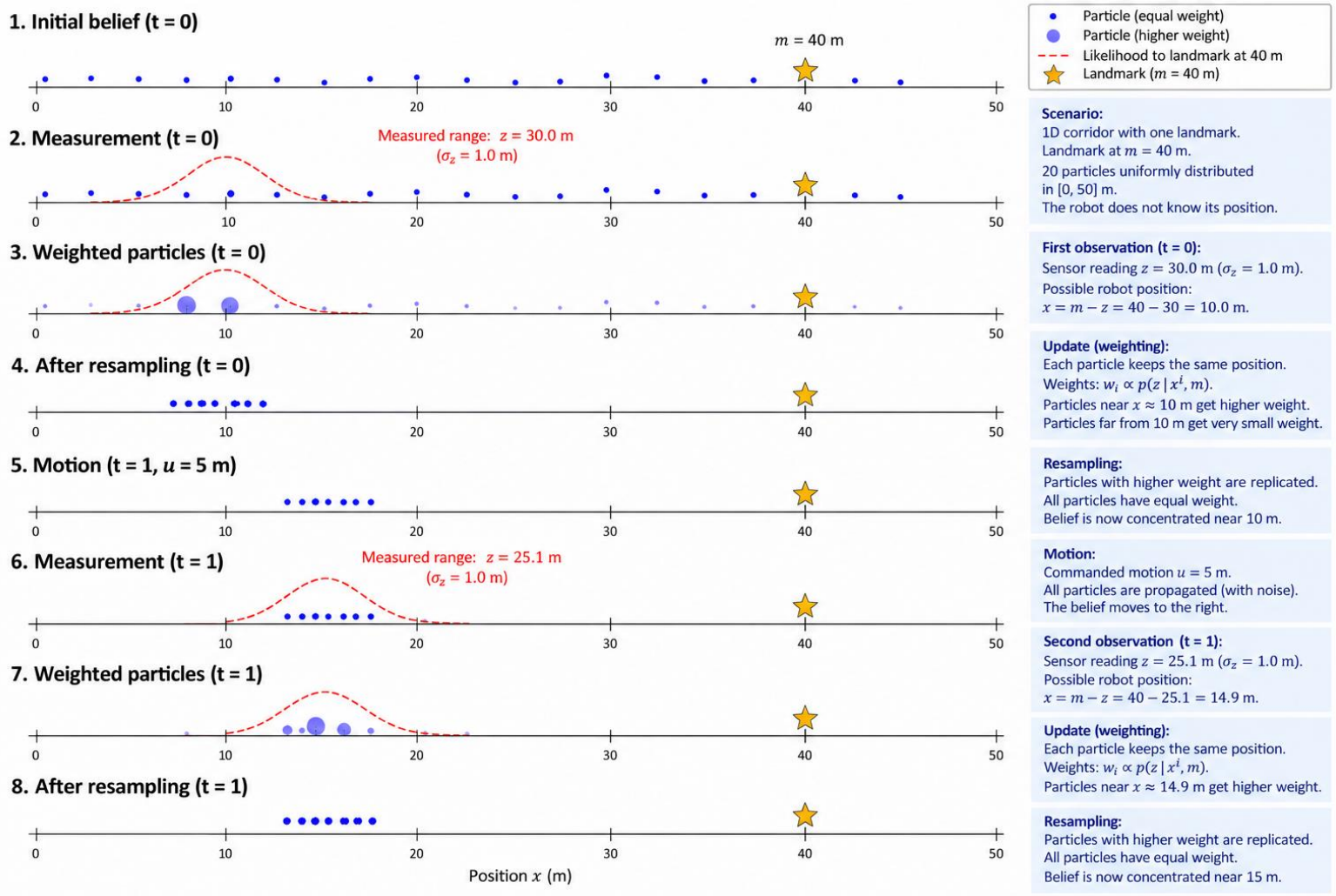


Particles with larger weights are more likely to be selected, possibly multiple times.

$$\left\{\mathbf{x}_t^{[i]}, \tilde{w}_t^{[i]}\right\}_{i=1}^{N} \xrightarrow{\text{resampling}} \left\{\mathbf{x}_t^{[i]}, 1/N\right\}_{i=1}^{N}.$$

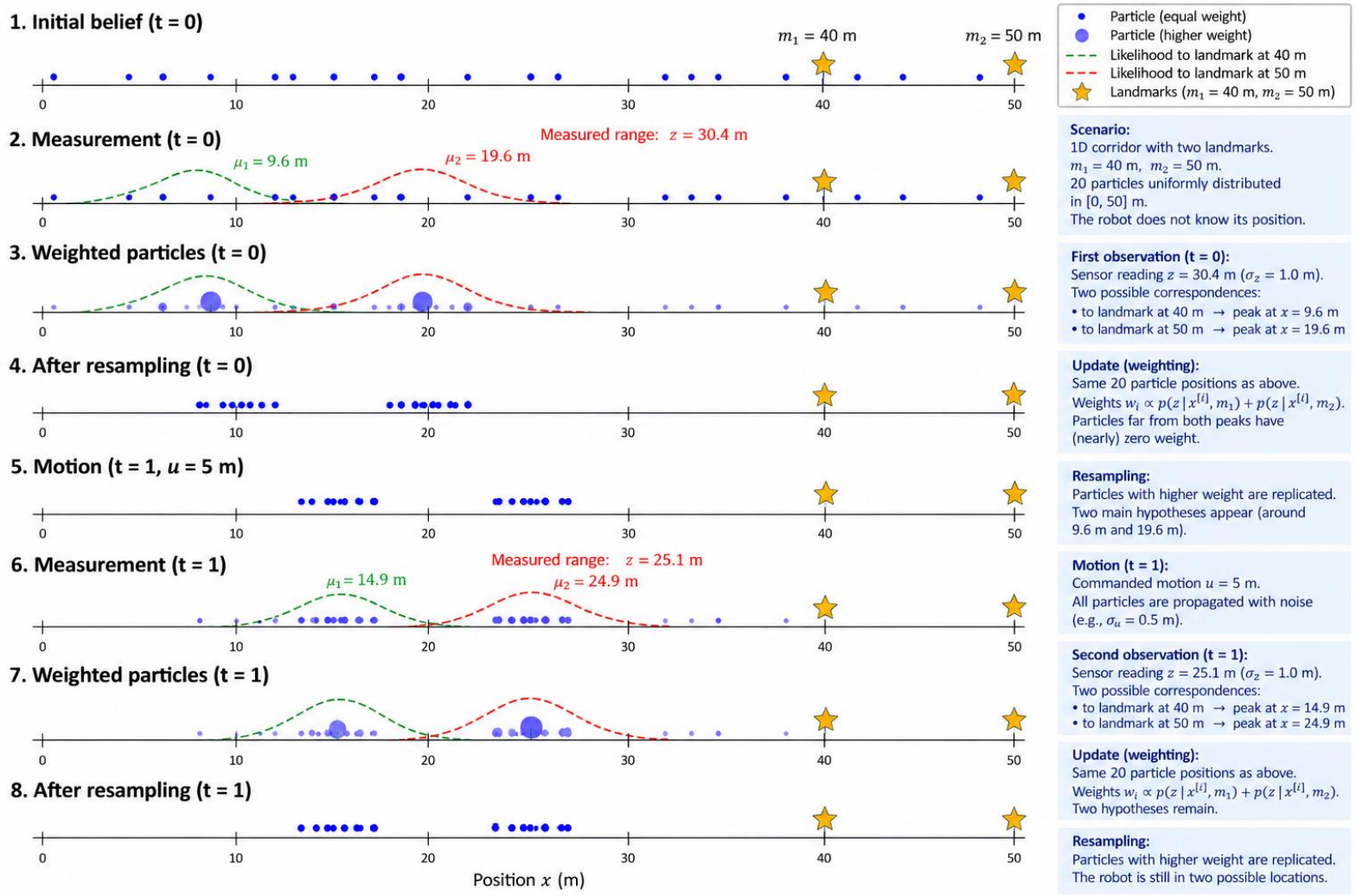
Example in 1D: Position tracking. A robot moves in a corridor with a range sensor that gets the distance to ONE landmark at the end of the corridor

At t= 0, the robot has no idea where it is in the corridor (samples from a uniform distribution [0,50])
particles = 20, **Motion**: 5 meters + noise; **Measurement:** true distance to the landmark (+ gaussian noise)



Example in 1D: Multiple hypothesis. A robot moves in a corridor with a range sensor that gets a distance to a landmark. It has no clue which one it is

At t= 0, the robot has no idea where it is in the corridor (samples from a uniform distribution [0,50])
particles = 20, **Motion**: 5 meters + noise; **Measurement:** true distance to the landmark (+ gaussian noise)



Particle Filter

Advantages

- Represent **non-Gaussian** and **multimodal** pose beliefs
- Allow **flexible motion and sensor** models
- Have a straightforward **basic algorithm**: predict, weight, resample

Disadvantages

- Need **enough particles** for an accurate approximation
- Resampling can reduce particle diversity and **discard plausible hypotheses**
- **Higher-dimensional states** often require more particles and computation

Summary - Robot Localization: Pose Estimation

Goal: Estimate the robot pose $\mathbf{x_t}$ from sensor observations and a map.

Least Squares Positioning: Find the pose $\hat{\mathbf{x}}$ that minimizes measurement errors.

- **Linear LS:** closed-form solution.
- **Nonlinear LS:** iterative solution → Gauss–Newton.
- **Weighted LS:** accounts for measurement uncertainty.

Point-set Registration: Estimate the rigid transformation $(R, \mathbf{t})$ aligning two point sets.

- **Known correspondences:** landmark registration.
- **Unknown correspondences:** ICP → alternate *matching + pose estimation*.
- Registration: against a *map* → **localization**;
  against a *previous scan* → **odometry**.

Summary - Robot Localization: Bayes Filtering

Goal: Estimate the complete pose belief

$$bel(\mathbf{x}_t) = p(\mathbf{x}_t \mid \mathbf{u}_{1:t}, \mathbf{z}_{1:t})$$

through a recursive cycle:

- **Prediction:** propagate the previous belief using the **motion model**.
- **Correction:** combine the prediction with the **sensor likelihood** using Bayes' rule.

Same Bayes principle — different belief representations:

Filter	Belief representation	Main feature
Grid Filter	Probability per cell	Discrete, arbitrary distributions
Kalman Filter	Gaussian $(\boldsymbol{\mu}, \Sigma)$	Exact for linear-Gaussian models
Extended KF	Gaussian $(\boldsymbol{\mu}, \Sigma)$	Nonlinear models → local linearization
Particle Filter	Weighted particles	Non-Gaussian & multimodal beliefs